VEL SOFTWARE SOLUTIONS · INTERNAL STANDARD OPERATING PROCEDURE

Software Delivery Handbook

From zero to delivery: the stage-by-stage guide every employee follows to research, design, build, test, secure, launch, sell and maintain a software product.

Document control	
Document	VEL-SOP-001 · Software Delivery Handbook
Version	2.0
Date	2026-11-03
Owner / author	V.LOKESHWARAN – VEL Software Solutions
Contact	velsoftwaresolutions.in@gmail.com
Audience	All employees, including beginners
Classification	Internal – confidential
Review	Every 6 months or after any major process change

Developed by **V.LOKESHWARAN** · velsoftwaresolutions.in@gmail.com · VEL Software Solutions

HOW TO USE

How to use this handbook

This handbook has one job: let a beginner finish a real software project the way a professional company does, **without having to guess**. Read Parts 0–5 once, then work stage by stage.

Contents	Page
How to use this handbook	2
The map – all stages on one page	4
How a software company works	6
Folder structures – where everything lives	8
Ground rules, Definition of Done, asking for help	10
12-week learning path	11
Plain-English glossary	12
Stage 0 – Get Ready	13
Stage 1 – Check the Idea (G1)	15
Stage 2 – Write What We Will Build (G2)	17
Stage 3 – Plan How It Will Work (G3)	19
Stage 4 – Design the Screens (G4)	21
Stage 5 – Set Up the Project (G5)	23
Stage 6 – Build It (G6)	25
Stage 7 – Test It (G7)	27
Stage 8 – Secure It (G8)	29
Stage 9 – Work Out Cost and Price (G9)	31
Stage 10 – Put It Online (G10)	33
Stage 11 – Show, Sell and Deliver (G11)	35
Stage 12 – Keep It Running (G12)	37
Appendix A – Forms and templates to fill	39
Appendix B – Worked cost, price and profit model	42
Appendix C – Quick-reference commands	48
Appendix D – The 20 standard documents	50
Appendix E – Source index and assumption register	51

Which part to read now?

If you are...	Read first	Then
New to the company (Day 1)	Parts 1–5 (about 2 hours)	Stage 0, then ask your Tech Lead for your first ticket
Starting a new product	Stage 1 → Stage 2	Do not open an editor until Gate G2 is signed
A developer on a ticket	Stage 5, Stage 6, Appendix A (commands)	Stage 7 when the feature is done
Preparing to launch	Stages 8, 9, 10	Never launch without G7, G8, G9, G10
After the customer goes live	Stage 11, Stage 12	Use the monthly checklists
Stuck	Section 3.3 (how to ask for help)	

How every stage page is laid out

1. **Banner**: stage number, phase and the gate (approval) at its end.
2. **Goal and quick facts**: when to start, who is involved, time, tools, skills.
3. **Step by step**: *what* to do, *how* to do it exactly, and the *proof* to save.

4. **Self-check**: tick every box before asking for approval.
5. **Sign-off**: who approves; signature lines.
6. **Common mistakes**, a **fictional example** (SlotWise), and **official sources** to learn more.

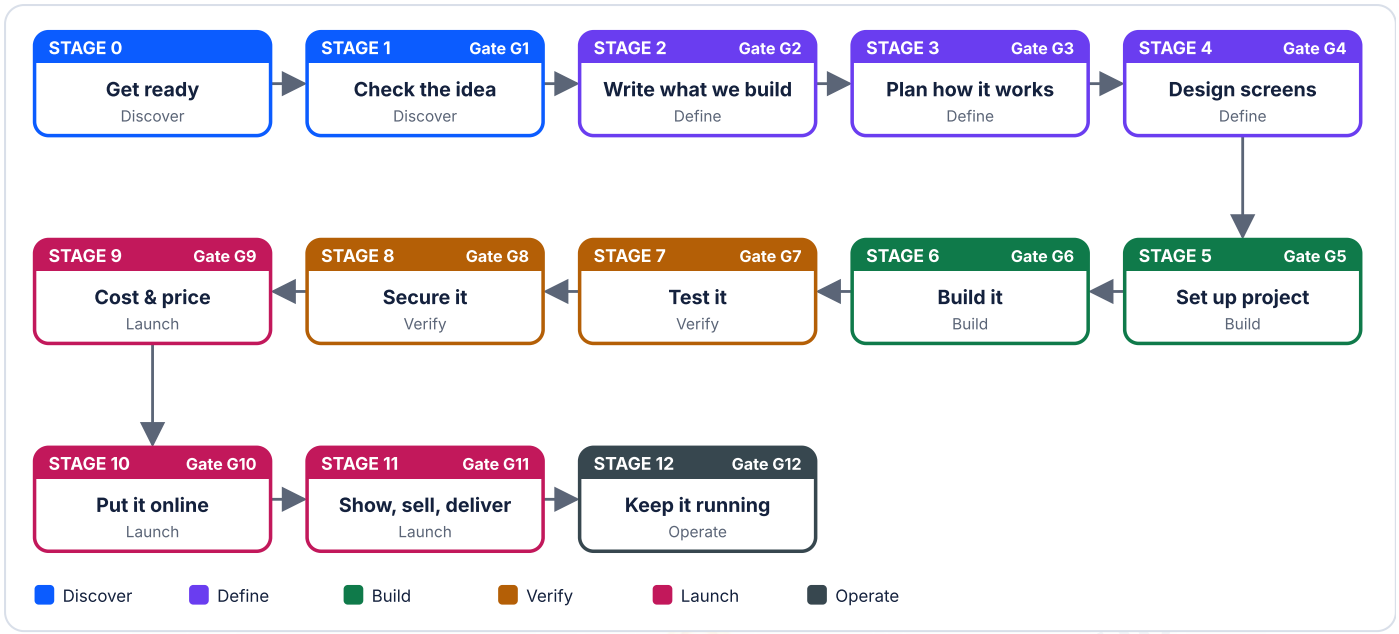
> **Important**
> Figures marked ASSUMPTION are planning placeholders. Prices, tax rates and legal duties change: always open the official source, confirm, and note the date. Nothing in this handbook is legal, tax or financial advice; confirm those with qualified professionals. Never claim a product is secure or production-ready without the evidence the gates require.

---

ed by V.LOKESHWARAN · velsoftwaresolutions.in@g
VEL
SOFTWARE SOLUTIONS

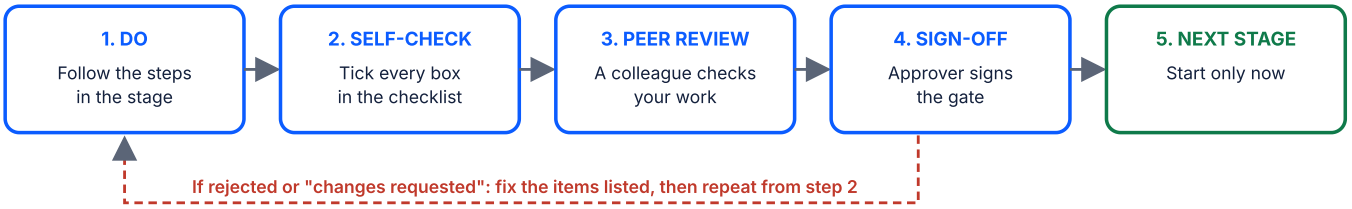
PART 0

The map – all stages on one page



Each stage ends with a numbered **gate**: a sign-off by a named person. Stage 0 has no gate. You may start Stage *n*+1 only when Gate G*n* is signed.

Every gate works the same way



Stages at a glance

Stage	Name	Phase	Goal in one line	Gate	Typical time
0	Get Ready	Discover	Have every account, tool and permission working so nothing blocks you later.	–	Day 1–2
1	Check the Idea	Discover	Prove that real people have this problem and will pay to solve it, before the company spends money building.	G1 – Opportunity Approval	2–3 weeks
2	Write What We Will Build	Define	Write down exactly what will be built, so everyone builds the same thing and later changes are controlled.	G2 – Scope Freeze	2–3 weeks
3	Plan How It Will Work	Define	Decide the technology and the design of the system and database in a way that is simple, secure and affordable.	G3 – Architecture Approval	1–2 weeks
4	Design the Screens	Define	Draw every screen and test it with real users before any code is written, because changing a drawing costs minutes and changing code costs days.	G4 – Design Approval	2–4 weeks
5	Set Up the Project	Build	Create one secure, automated project home so every developer starts from the same baseline.	G5 – Repository Ready	2–3 days
6	Build It	Build	Turn the frozen requirements into working software in small, tested, reviewed pieces with a demo every week.	G6 – Feature Complete	8–14 weeks (sample product)
7	Test It	Verify	Find the problems before customers do, with proof that every requirement works.	G7 – Quality Approved	2–3 weeks (overlaps Stage 6)
8	Secure It	Verify	Make sure we do not leak data, allow break-ins, or ship known vulnerabilities.	G8 – Security Approved	3–5 days

Stage	Name	Phase	Goal in one line	Gate	Typical time
9	Work Out Cost and Price	Launch	Know what the product truly costs to build and run, and set a price that leaves a profit after every cost.	G9 – Cost & Price Approved	1–2 weeks
10	Put It Online	Launch	Take the tested product from the repository to a real address, safely, with monitoring and a way back.	G10 – Go-Live Approved	2–4 days
11	Show, Sell and Deliver	Launch	Win paying customers honestly, onboard them well and hand the product over with written acceptance.	G11 – Customer Acceptance	2–8 weeks per customer
12	Keep It Running	Operate	Keep the service up, secure, supported and profitable, because subscriptions only continue if the product works every day.	G12 – Operations Accepted	Ongoing

velsoftwaresolutions.in@g
ed by V.LOKESHWARAN
VEL
SOFTWARE SOLUTIONS

PART 1

How a software company works

1.1 What a software company really is

A software company turns **a customer's problem** into **a working product** and then **keeps it working** while customers pay for it. Everything in this handbook exists to make that repeatable: the same steps, in the same order, with the same checks, whoever does the work. Corporate work is not about being clever alone; it is about being **predictable, documented and reviewed**.

1.2 Who does what (roles)

Role	In plain words	Typical output	You report to
Managing Director / Owner	Owns the company, money and final yes/no on big decisions	Approvals, budget, priorities	–
Product Owner (PO)	Speaks for the customer; decides *what* gets built and in what order	PRD, priorities, acceptance of work	Owner
Tech Lead	Decides *how* it is built; guards quality and security	Architecture, code reviews, technical approvals	Owner
Developer (frontend / backend / full-stack)	Writes and tests the code	Features, tests, documentation	Tech Lead
UI/UX Designer	Designs how the product looks and feels	Wireframes, designs, prototype	PO
QA Engineer	Tries to break the product before customers do	Test plans, bug reports, QA report	Tech Lead
DevOps Engineer	Runs servers, deployment, monitoring and backups	Pipelines, environments, uptime	Tech Lead
Support Lead	First contact for customers after launch	Tickets, onboarding, training	PO
Sales / Business Development	Finds and signs customers	Leads, demos, proposals	Owner

In a small startup one person can hold several roles. The **responsibilities** stay separate: the person who wrote the code never approves their own code.

1.3 A normal working day

Time	What happens	Why
09:30 – 09:45	**Stand-up**: each person says what they did yesterday, what they do today, and what blocks them (max 1 minute each)	Everyone sees problems early
09:45 – 13:00	**Focused work** on your ticket. Chat is for questions, not conversations; mute notifications if needed	Deep work produces quality
13:00 – 14:00	Lunch	
14:00 – 17:30	Focused work, code reviews you owe to others, meetings	Review is part of the job, not extra
17:30 – 17:45	**Update the board and timesheet**: move your ticket, log hours against Project ID + stage, write a 2-line note	Management can see progress without asking you
Friday 16:00	**Weekly demo + review**: show working software on staging; fill weekly report	Progress is proven by working software, not by words

Hours are an example. Use the actual working hours of your company, but keep the rhythm: stand-up, focus, review, update.

1.4 Meeting rhythm

Meeting	How often	Who	Output
Stand-up	Daily, 15 min	Whole team	Blockers list
Weekly review	Weekly, 30–45 min per project	Owner, PO, Tech Lead, team	Status, risks, decisions
Demo	Weekly	Team + PO (+ customer)	Accepted / rejected features
Retrospective	Every 2–4 weeks	Team	3 things to keep, 3 to change
Cost & KPI review	Monthly	Owner, PO, Tech Lead	Cost vs plan, MRR, churn, margin

1.5 How work flows: from request to done

1. Work starts as a **ticket** (an item on the board) written by the PO: what, why, acceptance criteria.

2. Board columns: **Backlog → To Do → In Progress → In Review → In Test → Done**. Only move a ticket when the exit rule of its column is true.
3. A ticket in **In Progress** has your name, a branch, and an estimate in hours. Nobody works on something without a ticket.
4. A ticket reaches **Done** only when it meets the **Definition of Done** (section 3).
5. New ideas during work are **not** built. They become a new ticket or a Change Request for the PO to decide.

1.6 Environments: where the software lives

Environment	Who uses it	Data	Rule
Local (your computer)	You	Fake / seed data	Break things freely
Staging (test copy online)	Team, PO, QA, customer demo	Fake or authorised data	Every merge lands here first
Production (the real thing)	Customers	Real customer data	**Never** edit directly; only deploy through the pipeline after approval

> **Golden rule of production**
> Real customer data is a legal and ethical responsibility. Never copy production data to your laptop, never paste it in chat, never use it in demos without written permission.

1.7 Documents, tickets and code: where truth lives

Thing	Lives in	Never in
Decisions, requirements, designs	Documents folder / repo `docs/`, versioned	Chat messages, memory
Tasks, bugs, progress	Board (tickets)	Private notes
Code	GitHub repository	Email, USB, personal cloud
Passwords and API keys	Company password manager and provider secret stores	Chat, email, code, screenshots
Time spent	Timesheet	Memory at month end

1.8 Professional communication

- **Write so someone can act without asking you again**: context, what you did, what you need, by when.
- Ask for help after **30 minutes** of honest effort. Bring: what you wanted, what you tried, the exact error text, a screenshot.
- Bad news early is a small problem; bad news late is a big one. Say 'I am blocked' or 'this will take longer' as soon as you know.
- Be polite and specific in code reviews. Review the code, never the person.
- Answer messages within the same working day; acknowledge long ones with 'seen, will reply by 4 pm'.

1.9 Confidentiality and conduct

- Customer data, source code, prices, contracts and plans are **confidential**. Do not share them outside the company.
- Use only company-approved tools and accounts for company work. Do not use personal email or personal cloud storage.
- Do not install pirated software; use only licences the company owns or free, permissively licensed software approved by the Tech Lead.
- Report any suspected security problem immediately to the Tech Lead, even if it was your mistake. Hiding it is the real offence.
- Credit: this system is owned by VEL Software Solutions; do not copy it out of the company.

PART 2

Folder structures – where everything lives

A tidy structure is how a team finds things at 3 pm on a Friday. Use these structures **exactly**. Folder names are lowercase with hyphens, no spaces.

2.1 Company shared drive

```
VEL-Software-Solutions/
├─ 00-company/              policies, templates, brand (logo), handbook
│   ├─ templates/           the 20 standard templates (copy, never edit the original)
│   └─ policies/            security, code of conduct, leave
├─ 01-clients/              one folder per customer
│   └─ acme-clinic/         contracts, SOWs, meeting notes, acceptance forms
├─ 02-projects/             one folder per project, named P001-slotwise
│   └─ P001-slotwise/
│       ├─ 01-research/     market report, competitor sheet, interview notes
│       ├─ 02-requirements/ PRD, input-output spec, scope freeze, change requests
│       ├─ 03-architecture/ diagrams, ERD, API list, threat model
│       ├─ 04-design/       wireframes, designs, prototype link, usability notes
│       ├─ 05-qa-security/  test plan, bug list, QA report, security report
│       ├─ 06-finance/      cost model, pricing model, invoices
│       ├─ 07-deployment/   domain/DNS report, deployment report, runbook
│       ├─ 08-sales/        demo script, proposal, demo recording
│       ├─ 09-handover/     manuals, training slides, acceptance form
│       └─ 10-maintenance/  monthly reports, incident reports
├─ 03-finance/              company-level accounts, tax, subscriptions
└─ 04-hr/                   timesheets, onboarding checklists (restricted access)
```

2.2 Your computer

```
~/work/                     (Windows: C:\work\)
├─ P001-slotwise/           the Git repository you cloned (see 2.3)
├─ training-sandbox/        practice repository
└─ scratch/                 throw-away experiments (never company data)
```

Keep code **outside** synced personal folders (Desktop, personal OneDrive/Google Drive). Real work is saved by **committing and pushing to GitHub**, not by copying folders.

2.3 Inside a project repository (read every line once)

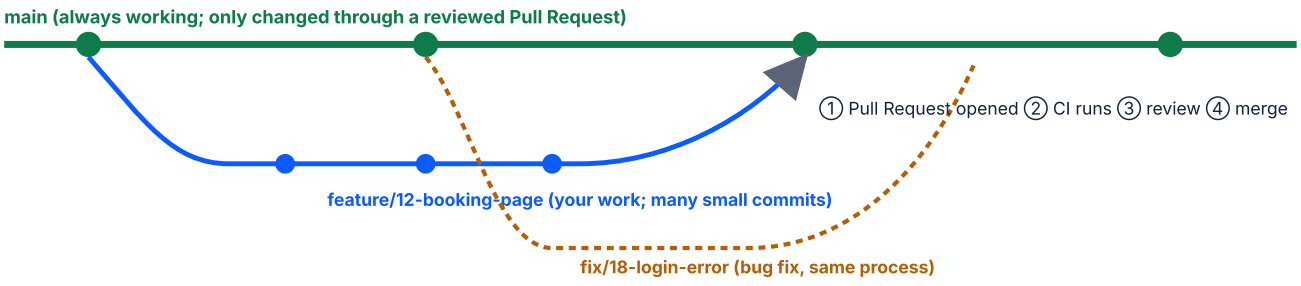
```
slotwise/
├─ apps/
│   ├─ web/                 the website users see (frontend)
│   └─ api/                 the server logic (backend) — may live inside web/ for small projects
├─ packages/
│   ├─ ui/                  reusable buttons, forms, tables
│   └─ shared/              types and helpers used by web and api
├─ docs/
│   ├─ requirements/        PRD, input-output spec
│   ├─ architecture/        diagrams, decisions (ADR-001.md ...)
│   ├─ database/            ERD, data dictionary
│   ├─ api/                 endpoint list / OpenAPI file
│   ├─ deployment/          deploy steps, runbook, rollback
│   └─ testing/             test plan, QA reports
├─ tests/
│   ├─ unit/                test one small function
│   ├─ integration/         test API + database together
│   └─ e2e/                 test a whole user journey in a browser
├─ infra/                   server/pipeline configuration files
├─ .github/
│   ├─ workflows/ci.yml     automatic checks on every Pull Request
│   ├─ ISSUE_TEMPLATE/      bug / feature forms
│   └─ pull_request_template.md
├─ .env.example             list of setting NAMES with fake values (committed)
├─ .env                     your real local settings (NEVER committed)
├─ .gitignore               files Git must ignore (node_modules, .env, builds)
├─ README.md                what this is and how to run it in 30 minutes
└─ CONTRIBUTING.md          how to work on it (branches, commits, reviews)
```

2.4 Naming rules

Thing	Rule	Example
Project ID	P + 3 digits, then short name	P001-slotwise
Branch	type/ticket-short-name	feature/12-booking-page · fix/18-login-error · hotfix/21-payment-crash
Commit message	type: what changed (present tense, ≤72 chars)	feat: add slot conflict check
Document file	ProjectID_Document_vMajor.Minor_YYYY-MM-DD	P001_PRD_v1.0_2026-11-03.pdf
Database table	lowercase, plural, snake_case	appointments, patients
Code files / components	As per the language convention used in the repo	BookingForm.tsx
Change Request	CR-ProjectID-number	CR-P001-004

Commit types: `feat` new feature · `fix` bug fix · `docs` documentation · `test` tests · `refactor` restructure without behaviour change · `chore` tooling.

2.5 Git workflow in one picture



PART 3

Ground rules, Definition of Done, asking for help

3.1 The ten rules

#	Rule	What it means for you
1	**Never work without a ticket.**	If it is not on the board, it is not approved work.
2	**Never commit passwords, keys or customer data.**	Use environment variables. If you commit one by accident, tell the Tech Lead at once; the secret must be replaced, not just deleted.
3	**Never push straight to main.**	Always branch → Pull Request → review → merge.
4	**Never edit production by hand.**	Only through the approved deployment pipeline.
5	**Never approve your own work.**	A second person reviews every change and every gate.
6	**Never guess a price, a DNS value or a legal clause.**	Open the official page, copy the exact value, save the link and date.
7	**Never skip a gate.**	No coding before G2; no launch before G7, G8, G10.
8	**Never change scope silently.**	New ideas go through a Change Request.
9	**Never claim 'done' without proof.**	Done = passes acceptance criteria + tested + reviewed + documented + demonstrated.
10	**Always log your time and update the board daily.**	It is how the company is paid and how projects are priced.

3.2 Definition of Done

Definition of Done – every task	Tick
Acceptance criteria all pass (shown, not claimed)	☐
Automatic tests written and passing	☐
Code reviewed and approved by a second person	☐
No new warnings from lint / security checks	☐
Merged through a Pull Request; branch deleted	☐
Works on staging (not only on your laptop)	☐
Documentation / README updated if behaviour changed	☐
Time logged and ticket moved to Done	☐

3.3 How to ask for help

Step	Action	Time
1	Re-read the ticket, the PRD section and this handbook; search the error text	up to 30 min
2	Ask a colleague; bring: goal, what you tried, exact error text, screenshot	next
3	Ask the Tech Lead (technical) or Product Owner (what/why)	if still blocked
4	Owner decides money, legal, customer-relationship questions	rarely

PART 4

12-week learning path for beginners

You do not need to know everything on Day 1. Spend about 5 working hours a week following this path while you do real tickets. Each week has a **proof** your Tech Lead can check.

When	Topic	Official / free source	Link	Proof you learned it
Week 1	Computers & terminal basics; Git	Pro Git book (free)	https://git-scm.com/book/en/v2	Clone, branch, commit, push, open a PR in the practice repo
Week 2	HTML, CSS, how the web works	MDN Learn web development	https://developer.mozilla.org/en-US/docs/Learn_web_development	Build a static page, responsive at 360 px
Week 3	JavaScript / TypeScript	MDN JavaScript; TypeScript Handbook	https://www.typescriptlang.org/docs/handbook/intro.html	Small functions with unit tests
Week 4	React	React Learn	https://react.dev/learn	Form component with validation
Week 5	Next.js	Next.js Learn / docs	https://nextjs.org/learn	Multi-page app with one API route
Week 6	Databases & SQL	PostgreSQL tutorial and docs	https://www.postgresql.org/docs/current/tutorial.html	Design 3 tables, write joins
Week 7	APIs & auth	MDN HTTP; OWASP Cheat Sheets	https://cheatsheetseries.owasp.org/	Login with hashed password, role check
Week 8	Testing	Playwright / framework docs	https://playwright.dev/docs/intro	One unit, one integration, one end-to-end test
Week 9	Security	OWASP Top 10, ASVS	https://owasp.org/www-project-top-ten/	Run the security checklist on your app
Week 10	Deploy, DNS, HTTPS	Cloudflare Learning Center; hosting docs	https://www.cloudflare.com/learning/dns/what-is-dns/	Deploy to staging with a custom subdomain
Week 11	Accessibility & performance	web.dev Learn; WCAG 2.2	https://web.dev/learn	Lighthouse score ≥ 90 on your page
Week 12	Business basics: SaaS metrics	This handbook, Appendix B	–	Compute break-even for a sample plan

Verify
Links are starting points to official documentation; open each, confirm it is current and record the date in your learning log. Learning time is working time: agree 5 hours per week with your Tech Lead.

PART 5

Plain-English glossary

Term	Plain-English meaning
API	A set of doors a program offers so other programs can ask it for data or actions.
Acceptance criteria	Plain 'Given/When/Then' sentences that say exactly when a feature counts as correct.
Backend	The hidden server part: rules, database access, security.
Frontend	What the user sees and clicks in the browser.
Branch	A separate copy of the code where you work without disturbing others.
CI (Continuous Integration)	Robot that runs checks on every change automatically.
Commit	A saved snapshot of your changes with a message.
CDN	Network of servers that deliver files quickly from near the user.
Database	Organised storage of data (tables of rows and columns).
Deploy	Putting a version of the software on a server where users can use it.
DNS	The internet's phone book: turns example.com into a server address.
Environment variable	A setting (like a password) kept outside the code.
ERD	Diagram of database tables and how they link.
Gate	A mandatory approval before the next stage starts.
HTTPS / TLS	Encrypted connection that protects data between browser and server.
Input validation	Checking data is allowed before using it.
Merge	Combining a branch into main.
Migration	A script that changes the database structure safely and repeatably.
MVP	Smallest product that delivers real value to a first customer.
MFA	Second proof of identity (app code) besides the password.
MRR / ARR	Monthly / Annual recurring revenue from subscriptions.
CAC	Cost to win one new paying customer.
Churn	Share of customers who cancel in a period.
Modular monolith	One application, organised inside into clear modules.
Pull Request (PR)	A request to merge your branch; where review happens.
RBAC	Role-based access control: what each role may do.
Repository (repo)	A project folder tracked by Git.
Rollback	Returning to the previous working version.
SaaS	Software sold as a subscription over the internet.
SLA	Written promise about service levels (uptime, response time).
Smoke test	Quick check that the main functions work after a deployment.
Staging	Online test copy of production.
Tenant	One customer organisation using a shared product.
Ticket	A tracked task or bug.
TAM / SAM / SOM	Total / reachable / winnable market.
Unit test	Automatic test of one small piece of code.
User story	'As a I want so that .'
Version tag	A label such as v1.2.0 on a release.
WCAG	International accessibility guidelines.
Webhook	A message another service sends your app when something happens.

STAGE 0 / 12

Get Ready

DISCOVER PHASE

No gate – readiness check

Goal: Have every account, tool and permission working so nothing blocks you later.

Start when	Who is involved	Time	Tools	Skills needed
You have joined, or been assigned a project, and your manager has given you a Project ID (for example P001).	You · Tech Lead (approves) · Owner (creates accounts)	Day 1–2	Password manager, authenticator app, VS Code, Git, Node.js LTS, browser, API client, database client	None. This stage teaches the basics.

Step by step

#	What to do	How to do it (exact)	Proof to save
1	**Read Parts 1–5 of this handbook**	Read slowly. Write down every word you do not understand and look it up in Part 5 (Glossary). About 2 hours.	Tick 'I have read the handbook' on your onboarding sheet
2	**Set up your company accounts**	Use only your company email. Create a password manager account; use a different long password for every service. Turn on **MFA** (two-factor) on email, GitHub, hosting and domain accounts using an **authenticator app** (not SMS). Store recovery codes in the company's shared vault, never in chat or on paper left on a desk.	Screenshot of MFA 'enabled' on each account (hide codes)
3	**Install the standard tools**	Download only from the official sites: VS Code, Git, Node.js (choose LTS), Postman or Bruno, DBeaver or pgAdmin. Then open a terminal and run `git --version`, `node --version`, `npm --version`. Each must print a version number.	Paste the 3 version outputs in your onboarding sheet
4	**Set up VS Code**	Install extensions: ESLint, Prettier, GitHub Pull Requests. In Settings turn on *Format on Save*. Learn 5 shortcuts: open file (Ctrl+P), search all (Ctrl+Shift+F), terminal (Ctrl+`), command palette (Ctrl+Shift+P), multi-cursor (Alt+Click).	VS Code opens a folder and formats a file on save
5	**Tell Git who you are and create an SSH key**	`git config --global user.name "Your Name"` then `git config --global user.email "you@company.com"`. Create a key: `ssh-keygen -t ed25519 -C "you@company.com"` (press Enter to accept defaults, set a passphrase). Copy the *.pub* file contents into GitHub → Settings → SSH keys. Test: `ssh -T git@github.com` should greet you by username.	Greeting message from the test command
6	**Get access to company systems**	Ask the Tech Lead for: GitHub organisation invite, the board (tickets), the shared drive (Part 2), the dashboard, the staging environment URL. Open each one and confirm you can see your project.	Access checklist ticked by Tech Lead
7	**Create your work folders**	Make `~/work/` (Windows: `C:\work\`) and inside it your project folder named with the Project ID. Never keep code on the Desktop or in personal cloud folders.	Folder exists; matches Part 2.2
8	**Start your timesheet**	Every day log: Project ID, stage number, hours, 1-line note. Do it before you leave, not at month end.	First day entry saved
9	**Do the practice task**	Clone the `training-sandbox` repository, create a branch `feature/0-your-name`, change one line in the README, commit, push, open a Pull Request and ask for review. This proves the full workflow works for you.	Link to your merged practice Pull Request

Self-check before you ask for approval

☐ MFA is on for every account
☐ I can run git, node and npm
☐ SSH test greets me by username
☐ I can open the board, drive, repo and dashboard
☐ My practice Pull Request was reviewed and merged
☐ I know where to log time

Sign-off: Tech Lead checks the onboarding sheet and your practice Pull Request.

Prepared by (name/date)	Reviewed by (name/date)	Approved by (name/date)

Common mistakes

Common mistake	What to do instead
Used personal email to sign up	Ask Owner to move the account to the company email and rotate the password
MFA by SMS only	Switch to authenticator app; keep recovery codes in the shared vault
'git' is not recognised	Close and reopen the terminal after installing; if still failing, reinstall and tick 'add to PATH'
Permission denied (publickey)	The public key was not added to GitHub, or you copied the wrong file (.pub, not private)

Example (fictional training product SlotWise – numbers are illustrative)
You are assigned P001-slotwise. By the end of Day 2 you can clone its repository, run it locally, and your first tiny Pull Request has been merged.

Learn more (official sources – verify the page and note the date)

- Pro Git book https://git-scm.com/book/en/v2
- GitHub docs: SSH https://docs.github.com/en/authentication/connecting-to-github-with-ssh

Stuck? Try 30 minutes alone → ask a colleague → ask the Tech Lead (technical) or Product Owner (scope). Bring the goal, what you tried, the exact error, a screenshot.

ed by V.LOKESHWARAN · velsoftwaresolutions.in
VEL SOFTWARE SOLUTIONS

STAGE 1 / 12

Check the Idea

DISCOVER PHASE

G1 – Opportunity Approval

Goal: Prove that real people have this problem and will pay to solve it, before the company spends money building.

Start when	Who is involved	Time	Tools	Skills needed
The Owner has given you an idea or a market to explore, and a Project ID.	Product Owner (leads) · Researcher / you · Owner (approves G1)	2–3 weeks	Spreadsheet, Google, G2, Capterra, Product Hunt, Reddit, Google Trends, phone/video calls, notes	Basic spreadsheet use, polite conversation.

Step by step

#	What to do	How to do it (exact)	Proof to save
1	**Write the problem in one sentence**	Use this template: **"[Who] struggle to [do what] because [reason], which costs them [time or money]."** Example: "Small clinic receptionists struggle to manage appointments on paper because calls are missed, which costs the clinic patients." If you cannot fill all four blanks, you do not understand the problem yet.	Sentence at top of the research report
2	**Choose the customer segment and name three people**	Write who the **buyer** (pays), the **user** (uses daily) and the **decision-maker** (says yes) are. They are often three different people. Describe the ideal customer in 5 lines: size, location, tools they use now, budget owner.	Ideal Customer Profile (ICP) box
3	**Find 5–10 competitors**	Search Google; browse G2, Capterra, Product Hunt categories; ask in relevant Reddit/professional groups. For each product open its website, features page and **pricing page**. Copy into a sheet: name, link, **date you visited**, target customer, plans and prices, free trial, main features, review score and number of reviews, weaknesses customers complain about. **Never type a price from memory.**	Competitor sheet + dated screenshots
4	**Compare and find the gap**	Make a table: features down the side, competitors across. Mark what each does well/badly. Underline what customers want but nobody does well. That gap is your reason to exist.	Comparison table + 3-line 'our difference'
5	**Interview at least 10 target customers**	Ask permission first. Ask about **what they did last time**, not opinions. Five questions: (1) Tell me about the last time this problem happened. (2) What did you do about it? (3) What does it cost you? (4) What do you use today and what do you pay? (5) If a tool solved this, what would you pay? Write notes in the Customer Discovery template.	10 interview notes with date, role, quotes, consent
6	**Test willingness to pay**	Ask the buyer: 'If we offered X at ₹Y per month, would you do a paid pilot?' Record the answer exactly. 'Sounds nice' is **not** a yes. A pilot commitment or letter of intent is a yes.	Count of pilot commitments
7	**Estimate the market (bottom-up)**	**TAM** = number of possible customers × yearly price per customer. **SAM** = the part you can really reach (city, size, language). **SOM** = what you can win in 3 years with your sales capacity. Get the number of customers from a government or industry source and save the link + date. If none exists, write **ASSUMPTION** next to it. Never invent a number.	TAM/SAM/SOM sheet with sources
8	**Estimate how hard selling will be**	Note how customers buy (word of mouth, search, demos), how long a deal takes (days/weeks), and what a customer costs to find (CAC range). This feeds Stage 9.	Sales-cycle paragraph
9	**List ways to earn recurring revenue**	Subscription tiers, extra usage fees, onboarding fee, support plans, add-on modules.	Revenue ideas list
10	**Write the Market Research Report (2–4 pages)**	Sections: 1 Problem sentence · 2 Customer (ICP) · 3 Evidence from interviews · 4 Competitors table + gap · 5 Market size with sources · 6 Sales difficulty · 7 Revenue options · 8 Risks · 9 Recommendation: **GO / NO-GO / CHANGE DIRECTION** with reasons.	Report PDF saved in 01-research
11	**Submit Gate G1**	Upload the report to the dashboard and request approval. The Owner reads it and decides. Do not start Stage 2 until G1 is signed.	G1 signed

Self-check before you ask for approval

☐ Problem sentence has all four blanks filled
☐ At least 5 competitors with link + date
☐ At least 10 customer interviews with notes
☐ Every number has a source link or is marked ASSUMPTION

☐ Buyer, user and decision-maker named
☐ At least 3 of 10 prospects confirmed the problem is a top-3 priority
☐ A clear GO / NO-GO recommendation is written

Sign-off: Owner signs G1. Kill rule: if fewer than 3 of 10 prospects confirm the pain, or nobody will pay near the break-even price, the answer is NO-GO.

Prepared by (name/date)	Reviewed by (name/date)	Approved by (name/date)

Common mistakes

Common mistake	What to do instead
Only talking to friends	Interview strangers who match the ICP
Copying competitors' features	Start from the customer's pain, then pick features
'The market is huge'	Use bottom-up: customers × price
Prices from blogs	Use the competitor's own pricing page; note the date

Example (fictional training product SlotWise – numbers are illustrative)
SlotWise: problem sentence written; 8 competitor pages recorded with dates; 10 clinic owners interviewed; 4 agreed to a paid pilot → recommendation GO, Owner signs G1.

Learn more (official sources – verify the page and note the date)

- G2 (software reviews) https://www.g2.com/
- Capterra https://www.capterra.com/
- Google Trends https://trends.google.com/

Stuck? Try 30 minutes alone → ask a colleague → ask the Tech Lead (technical) or Product Owner (scope). Bring the goal, what you tried, the exact error, a screenshot.

STAGE 2 / 12

Write What We Will Build

DEFINE PHASE

G2 – Scope Freeze

Goal: Write down exactly what will be built, so everyone builds the same thing and later changes are controlled.

Start when	Who is involved	Time	Tools	Skills needed
G1 is signed.	Product Owner (writes) · Tech Lead (checks feasibility) · QA (checks testability) · Customer (approves)	2–3 weeks	Documents, spreadsheet, Figma (journeys), PRD and Input-Output templates	Clear writing, basic spreadsheet.

Step by step

#	What to do	How to do it (exact)	Proof to save
1	**Copy the PRD template**	From `00-company/templates/` into `02-requirements/`. Rename with the naming rule: `P001_PRD_v0.1_date`.	File created
2	**Write purpose and users**	Purpose: 3 sentences. Then a table of **roles** (e.g. Owner, Receptionist, Doctor, Patient, Platform Admin) with a one-line description each.	Purpose + roles table
3	**List modules and user stories**	A module is a group of features (Booking, Reminders, Patients, Billing, Reports, Admin). For each feature write: **"As a [role] I want [action] so that [benefit]."**	Module list with stories
4	**Make the permission matrix**	Rows = actions (create, view, edit, delete, export), columns = roles. Put ✓ or ×. The developers will turn this into server checks.	Permission table
5	**Write user journeys**	For each role write numbered steps from opening the app to reaching their goal. Example: 1 Receptionist logs in → 2 opens calendar → 3 picks free slot → 4 enters patient → 5 saves → 6 sees confirmation.	One journey per role
6	**Write the Input specification**	One spreadsheet row per field: **name · type · unit · required? · default · validation rule · minimum · maximum · where the data comes from · database column**. Example: *patient_phone · text · international format · yes · none · must match +country code, 8–15 digits · – · booking form · patients.phone*.	Input sheet complete
7	**Write the Output specification**	One row per screen, report, email or API response: **name · format · how it is calculated · decimal places · where shown · export format (PDF/CSV) · what happens on error**. Example: No-show rate = no-shows ÷ (completed + no-shows), 1 decimal, shown on Reports, export CSV, if denominator is 0 show 0.0%.	Output sheet complete
8	**Write acceptance criteria**	For each feature at least two scenarios in **Given / When / Then** form: one normal, one error. Example: Given slot 10:15 is taken, When another patient books 10:15, Then the system refuses and offers the next three free slots.	Criteria under each feature
9	**Add measurable non-functional requirements**	Numbers, not adjectives: page loads in under 3 seconds on mobile data; API answers in under 500 ms for 95% of requests; data backed up daily; works on the last two versions of Chrome, Edge, Safari, Firefox; layout works from 360 px width.	NFR table
10	**Create sample input → output scenarios**	At least 3 worked examples with real-looking fake numbers; QA will use them as tests.	Scenario table
11	**Review and freeze**	Hold a review meeting with the Tech Lead, QA and customer. Fix every comment. Stamp the documents **v1.0** with date and get signatures. From now on every change needs a **Change Request** (Appendix A) with: ID, reason, technical impact, cost impact, time impact, security impact, customer approval, PO approval.	G2 signed; scope frozen

Self-check before you ask for approval

☐ All roles and permissions written
☐ Every input and output is specified in the sheets
☐ Every feature has at least 2 acceptance criteria
☐ Out-of-scope list written
☐ Numbers (not words) in non-functional requirements
☐ Sample scenarios included
☐ Customer and Product Owner have signed

Sign-off: Product Owner and Customer sign G2. After this, nobody writes code until Stage 5 is complete.

Prepared by (name/date)	Reviewed by (name/date)	Approved by (name/date)

Common mistakes

Common mistake	What to do instead
'Fast', 'easy', 'modern'	Replace with numbers
Forgetting error cases	Every feature gets one 'what if it goes wrong' scenario
Adding features after freeze	Open a Change Request; the PO decides with cost and time impact

Example (fictional training product SlotWise – numbers are illustrative)
SlotWise: 6 modules, 31 user stories, 4 roles, 22 input fields, 9 outputs, 74 acceptance criteria, signed as v1.0.

Learn more (official sources – verify the page and note the date)

- OWASP ASVS (security requirements) https://asvs.owasp.org/

Stuck? Try 30 minutes alone → ask a colleague → ask the Tech Lead (technical) or Product Owner (scope). Bring the goal, what you tried, the exact error, a screenshot.

Developed by V.LOKESHWARAN · velsoftwaresolutions.in@gmail.com · verify all ASSUMPTIONS before use

STAGE 3 / 12

Plan How It Will Work

DEFINE PHASE

G3 – Architecture Approval

Goal: Decide the technology and the design of the system and database in a way that is simple, secure and affordable.

Start when	Who is involved	Time	Tools	Skills needed
G2 is signed.	Tech Lead (owns) · Senior developer (reviews) · DevOps (hosting/cost input)	1–2 weeks	draw.io or Excalidraw, ERD tool (dbdiagram.io), spreadsheet, document template	Basic understanding of web apps and tables.

Step by step

#	What to do	How to do it (exact)	Proof to save
1	**Read the PRD and list technical questions**	How many users? What sensitive data (names, phones, health, payments)? Which outside services (email, SMS, payment)? Any legal duties? Write answers; they drive every choice.	Question list answered
2	**Start from the company default stack**	Default: **TypeScript, React/Next.js, Node.js, PostgreSQL, an ORM with migrations**, hosted on a managed platform. Use something else only if the Tech Lead approves with a written reason.	Stack decision line
3	**Compare two architecture options**	Score each 1–5 on cost to build, cost to host, cost to maintain, scalability, security, performance, availability of developers, lock-in, integration fit. Multiply by weights (add to 100%). Start with a **modular monolith** (one app, clear modules inside); choose microservices only with written proof you need them.	Scoring table + winner
4	**Draw the system diagram**	Boxes: Browser → Web app → API → Database; plus Email service, SMS/WhatsApp, Payment gateway, File storage, Background jobs, Monitoring. Arrows show who talks to whom.	System diagram file in 03-architecture
5	**Design the database (ERD)**	List the 'things' (patients, appointments, doctors) – each becomes a **table**. For each table list columns and their type. Every table gets `id`, `created_at`, `updated_at`; tables that belong to a customer get `tenant_id`. Draw lines for relationships (one doctor has many appointments). Never store passwords in readable form.	ERD + data dictionary (table, column, type, meaning)
6	**List the API endpoints**	Table: method (GET read, POST create, PUT/PATCH change, DELETE) · path (e.g. `/api/appointments`) · who may call · input · output · possible errors. Use nouns in plural, versions like `/api/v1/`.	API list in docs/api
7	**Design login and roles**	How users sign in; passwords stored hashed using a trusted library (never invent your own); sessions or tokens expire; role checks happen on the **server**, never only in the screen.	Auth design page
8	**Write a one-page threat model**	Table: what could go wrong (someone sees another customer's data, guesses passwords, spams the booking page, steals a file) → how we stop it → how we test it.	Threat table
9	**Plan environments, backups and hosting**	Local, staging, production; each with separate database and secrets. Backups: how often, how long kept, who restores. Ask DevOps for hosting options and cost (feeds Stage 9).	Infrastructure diagram
10	**Write the Architecture Document and decisions**	Combine diagrams, ERD, API list, auth, threat model and a short 'Decision record' per big choice (what, why, alternatives). Present to the Tech Lead and Owner.	G3 signed

Self-check before you ask for approval

☐ Two options scored and the winner explained
☐ Diagrams match the PRD (nothing missing, nothing extra)
☐ Every table has id, created_at, updated_at; tenant data has tenant_id
☐ API list covers every PRD feature
☐ Login, roles and secrets handling written
☐ Backups and restore plan written
☐ Threat model has at least 8 rows

Sign-off: Tech Lead and Owner sign G3. Changing the stack later needs a written decision record and a Change Request.

Prepared by (name/date)	Reviewed by (name/date)	Approved by (name/date)

Common mistakes

Common mistake	What to do instead
Choosing technology because it is popular	Use the scoring table
One giant table for everything	Split into tables per 'thing' and link them
Role checks only in the screen	Always enforce on the server

Example (fictional training product SlotWise – numbers are illustrative)
SlotWise: Option A (Next.js + Node/TS modular monolith + PostgreSQL on managed hosting) scored 3.80 vs Option B 3.60; chosen.

Learn more (official sources – verify the page and note the date)

- PostgreSQL tutorial https://www.postgresql.org/docs/current/tutorial.html
- OWASP Top 10 https://owasp.org/www-project-top-ten/

Stuck? Try 30 minutes alone → ask a colleague → ask the Tech Lead (technical) or Product Owner (scope). Bring the goal, what you tried, the exact error, a screenshot.

VEL SOFTWARE SOLUTIONS

STAGE 4 / 12

Design the Screens

DEFINE PHASE

G4 – Design Approval

Goal: Draw every screen and test it with real users before any code is written, because changing a drawing costs minutes and changing code costs days.

Start when	Who is involved	Time	Tools	Skills needed
G3 is signed (or in parallel once G2 is signed, as agreed by the PO).	UI/UX Designer (leads) · Product Owner · Frontend developer · Customer	2–4 weeks	Figma, pen and paper, contrast checker (WebAIM), Lighthouse	Basic design sense; Figma is taught in-house.

Step by step

#	What to do	How to do it (exact)	Proof to save
1	**Make the sitemap**	A tree of every page: Login → Dashboard → Calendar → New booking → Patients → Reports → Settings. Take it from the user journeys in the PRD.	Sitemap
2	**Draw the user flows**	For each journey draw boxes and arrows with decisions ('slot free?'). Check every PRD journey appears.	Flow diagrams
3	**Sketch low-fidelity wireframes**	Grey boxes only: no colour, no pictures. Place navigation, titles, forms, tables, buttons. Review with the PO. Wireframes are cheap; change them freely.	Wireframes for every page
4	**Define the design system**	Choose: colours (primary, text, background, success, warning, error), fonts and sizes, spacing scale (4/8/16/24 px), corner radius. Check text contrast is at least **4.5:1** (WCAG AA) with a contrast checker. Create reusable components: button, input, select, table, modal, toast message.	Design system page in Figma
5	**Design the high-fidelity screens**	Apply the design system to each wireframe. For each screen also design the **empty state** (no data yet), **loading**, **error**, and **success** states. Avoid heavy animation and decorative gradients.	Hi-fi screens with all states
6	**Design the mobile layouts**	Design at 360 px, 768 px and 1280 px. Buttons at least 44×44 px for touch. Check text is readable without zoom.	Responsive frames
7	**Build a clickable prototype**	Link screens in Figma so a user can click through the main journey.	Prototype link
8	**Test with 5 target users**	Give each user 5 tasks ('book a patient for 10:15'). Do not help. Note: task finished? how long? where confused? quotes. Fix every problem that stopped 2 or more users.	Usability report
9	**Check accessibility basics**	Everything reachable by keyboard; labels on inputs; colour never the only signal; alt text for images.	Accessibility checklist ticked
10	**Hand over to developers**	Name components consistently, export icons/images, add notes for behaviour. Present to PO and customer and get approval.	G4 signed

Self-check before you ask for approval

☐ Every PRD journey has screens
☐ Empty, loading, error and success states designed
☐ Contrast at least 4.5:1 for text
☐ Mobile layouts done
☐ Usability test with 5 users completed and fixes made
☐ Customer approved the prototype

Sign-off: Product Owner and Customer sign G4. Design changes afterwards go through change control.

Prepared by (name/date)	Reviewed by (name/date)	Approved by (name/date)

Common mistakes

Common mistake	What to do instead
Designing only the happy path	Add error and empty states for every screen
Tiny buttons on mobile	Minimum 44×44 px

Common mistake	What to do instead
Using real customer data in designs	Use fake data

Example (fictional training product SlotWise – numbers are illustrative)
SlotWise: 14 screens × 3 sizes; 5 receptionists tested; the 'next free slot' button was moved after 3 of 5 missed it.

Learn more (official sources – verify the page and note the date)

- WCAG 2.2 https://www.w3.org/TR/WCAG22/
- web.dev Learn https://web.dev/learn

Stuck? Try 30 minutes alone → ask a colleague → ask the Tech Lead (technical) or Product Owner (scope). Bring the goal, what you tried, the exact error, a screenshot.

ed by V.LOKESHWARAN · velsoftwaresolutions.in@g
VEL SOFTWARE SOLUTIONS

STAGE 5 / 12

Set Up the Project

BUILD PHASE

G5 – Repository Ready

Goal: Create one secure, automated project home so every developer starts from the same baseline.

Start when	Who is involved	Time	Tools	Skills needed
G2, G3 and G4 are signed.	Tech Lead (owns) · DevOps (CI and environments) · All developers (follow)	2–3 days	GitHub, VS Code, terminal, Node.js, CI (GitHub Actions)	Stage 0 completed.

Step by step

#	What to do	How to do it (exact)	Proof to save
1	**Create the private repository**	GitHub → organisation → New repository. Name = project name in lowercase (`slotwise`). Visibility **Private**. Add a short description. Tick 'Add README'.	Repository link
2	**Clone it to your work folder**	`cd ~/work` then `git clone git@github.com:ORG/slotwise.git` then `cd slotwise`.	Folder exists on your computer
3	**Create the standard folders**	`mkdir -p apps/web apps/api packages/ui packages/shared docs/requirements docs/architecture docs/database docs/api docs/deployment docs/testing tests/unit tests/integration tests/e2e infra .github/workflows .github/ISSUE_TEMPLATE`. Git ignores empty folders, so add an empty `.gitkeep` file inside each one you want to keep.	Tree matches Part 2.3
4	**Add .gitignore and .env.example**	`.gitignore` must list at least: `node_modules/`, `.env`, `.env.*` (except .env.example), build output folders, OS files. `.env.example` lists every setting **name** with a fake value, e.g. `DATABASE_URL=postgres://user:password@localhost:5432/slotwise`.	Both files committed; real .env ignored
5	**Write README.md and CONTRIBUTING.md**	README: what this is, requirements, **exact steps to run it locally** (install, copy .env.example to .env, create database, run migrations, start). CONTRIBUTING: branch names, commit rules, Pull Request rules, how to run tests. Ask a new colleague to follow your README; fix every place they get stuck.	A colleague runs the app in under 30 minutes
6	**Create the app skeleton**	If the Tech Lead has a company starter template, use it. Otherwise inside `apps/web` run the official generator `npx create-next-app@latest` and accept the TypeScript and ESLint options. Make sure `npm run dev`, `npm run lint`, `npm test` and `npm run build` exist and pass.	All four commands succeed
7	**Protect the main branch**	GitHub → repository Settings → Branches (or Rules) → add rule for `main`: require a Pull Request, require at least 1 approval, dismiss stale approvals, require status checks to pass, require branch up to date, block force pushes, block deletion.	Screenshot of the rule
8	**Add the automatic checks (CI)**	Create `.github/workflows/ci.yml` with the file shown below. It runs lint, tests, build and a dependency audit on every Pull Request. Mark it as a required check in the branch rule.	Green check on a test PR
9	**Turn on security features**	Settings → Code security: enable Dependabot alerts and updates, secret scanning with push protection, and code scanning. These stop leaked keys and flag vulnerable libraries.	Features show 'Enabled'
10	**Prepare three environments**	**Local**: your `.env`. **Staging** and **Production**: separate databases, separate secrets, stored in the hosting platform's secret settings (never in the repo). Record only the *names* of the settings in docs/deployment/environments.md.	Environment matrix document
11	**Run a dry-run Pull Request**	Create `feature/0-ci-check`, change the README, push, open a PR, watch CI turn green, get a review, merge. Tag the first version: `git tag v0.1.0` then `git push --tags`.	Merged PR + tag v0.1.0

ci.yml (copy exactly, then adapt commands to your project)

```
name: CI
on:
  pull_request:
  push:
    branches: [main]
jobs:
  checks:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm
      - run: npm ci
      - run: npm run lint
      - run: npm test
      - run: npm run build
      - run: npm audit --audit-level=high
```

Self-check before you ask for approval

☐ Repository is private
☐ main is protected and requires review + passing checks
☐ CI is green on a test PR
☐ No secrets in the repo; .env is ignored
☐ Dependabot and secret scanning are on
☐ Staging and production are fully separate
☐ README works for a new person

Sign-off: Tech Lead signs G5 after the dry-run Pull Request passes everything.

Prepared by (name/date)	Reviewed by (name/date)	Approved by (name/date)

Common mistakes

Common mistake	What to do instead
Committed .env by mistake	Tell the Tech Lead immediately. Change every password/key in it. Remove it from the repo history with Tech Lead help
Branch rule not enforced for admins	Tick 'include administrators'
CI passes locally but fails on GitHub	Different Node version or missing environment variable; copy the CI error text and compare

Example (fictional training product SlotWise – numbers are illustrative)
SlotWise: repository slotwise created, structure committed, CI green, v0.1.0 tagged, G5 signed in 2 days.

Learn more (official sources – verify the page and note the date)

- GitHub: protected branches https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches
- GitHub Actions https://docs.github.com/en/actions
- Conventional Commits https://www.conventionalcommits.org/

Stuck? Try 30 minutes alone → ask a colleague → ask the Tech Lead (technical) or Product Owner (scope). Bring the goal, what you tried, the exact error, a screenshot.

STAGE 6 / 12

Build It

BUILD PHASE

G6 – Feature Complete

Goal: Turn the frozen requirements into working software in small, tested, reviewed pieces with a demo every week.

Start when	Who is involved	Time	Tools	Skills needed
G5 is signed and tickets exist for the first module.	Developers (build) · Tech Lead (reviews) · QA (prepares tests) · Product Owner (accepts)	8–14 weeks (sample product)	VS Code, Git, terminal, framework, database client, API client, board	Basic coding in the chosen language; Parts 2–4 read.

Step by step

#	What to do	How to do it (exact)	Proof to save
1	**Take one small ticket**	Pick the top ticket in **To Do**. It should take one day or less; if bigger, ask the PO/Tech Lead to split it. Move it to **In Progress** and write your time estimate.	Ticket assigned to you
2	**Make a branch**	`git switch main`, `git pull`, then `git switch -c feature/12-booking-form`. One ticket = one branch.	Branch visible on GitHub
3	**Write the test first or alongside**	Turn the ticket's acceptance criteria into automatic tests. Run them: they fail. That is correct; they will pass when you finish.	Failing tests that describe the goal
4	**Write the smallest code that works**	Keep functions short; use clear names; validate every input **on the server**; read secrets only from environment variables; never trust data from the browser; handle errors and show friendly messages.	Code compiles; tests pass
5	**Run the checks locally**	`npm run lint`, `npm test`, `npm run build`. Fix everything before pushing. Run the app and try it like a user would.	All three pass
6	**Commit often with clear messages**	`git add .` (check `git status` first so you do not add .env) then `git commit -m "feat: add slot conflict check"`. Small commits are easy to review and undo.	Meaningful commit history
7	**Push and open a Pull Request**	`git push -u origin feature/12-booking-form`. On GitHub open a PR; fill the template: what changed, why, ticket link, how to test, screenshots, checklist. Assign a reviewer.	PR link on the ticket; ticket in **In Review**
8	**Respond to review**	Reviewer comments are about the code, not you. Fix each comment with a new commit or explain politely why not. Do not merge until approved and CI is green.	Approval + green checks
9	**Merge and clean up**	Use 'Squash and merge'. Delete the branch. The change deploys to **staging** automatically. Open staging and verify your feature works there.	Feature works on staging
10	**Update documentation**	If behaviour, settings or API changed, update README / docs in the same PR. Move the ticket to **In Test** (QA) then **Done** when it meets the Definition of Done.	Docs updated
11	**Log time and show the weekly demo**	Every Friday show working features on staging to the PO. Accepted features are ticked; rejected ones go back with reasons. Compare actual hours with estimate; tell the Tech Lead if you are over 20%.	Demo notes; timesheet complete
12	**Finish the module list and request G6**	When all in-scope modules (list below) are accepted by the PO and no Critical/High bugs are open, request G6 'Feature complete'. No new features after this; only fixes.	G6 signed

Standard build order (follow top to bottom)

#	Module	What to build	Done when
1	Database & migrations	Create tables from the ERD as migration files	Fresh database builds from migrations; rollback works
2	Backend skeleton	Project layout, config from environment variables, error handler	App starts; health endpoint returns OK
3	Authentication	Sign-up/login, hashed passwords, sessions	Wrong password rejected; session expires
4	Authorisation (roles)	Server-side checks from the permission matrix	Each role blocked from forbidden actions (tested)
5	API endpoints	One endpoint at a time from the API list, with input validation	Valid input works; invalid input returns clear error
6	Frontend skeleton	Layout, navigation, design tokens, shared components	All pages reachable; responsive at 360 px

#	Module	What to build	Done when
7	UI components	Forms, tables, dialogs from the design system	Match approved designs incl. empty/loading/error
8	Business logic	Rules and calculations from the Output spec	Sample scenarios give exactly the expected numbers
9	Connect frontend and API	Replace fake data with real calls; handle failures	Works end-to-end on staging
10	File upload & storage	Safe upload: type and size limits, private storage	Bad file types rejected; files not publicly listable
11	Email & notifications	Templates, queue, retry, unsubscribe where required	Test emails delivered; failures logged
12	Reports & exports	PDF/CSV per Output spec	Totals match hand-calculated examples
13	Payments (if in scope)	Gateway checkout, webhooks verified by signature	Test-mode payments update subscription state
14	Admin dashboard	Tenant/user management, plan limits, audit log	Only Platform Admin can open it
15	User dashboard	Key numbers and shortcuts	Loads in under 3 s on mobile data
16	Error handling	Friendly messages to users, details to logs	No stack traces shown to users
17	Logging & monitoring hooks	Structured logs without personal data or secrets	Search a request id and see its full path
18	Documentation	README, API docs, runbook	New developer runs the app in 30 minutes

Self-check before you ask for approval

☐ Each ticket has a branch, a PR, a review and green checks
☐ Every input is validated on the server
☐ No secrets or customer data in code or logs
☐ Tests exist for business rules
☐ Docs updated with each change
☐ Weekly demos held; PO accepted every in-scope module

Sign-off: Product Owner accepts all in-scope modules; Tech Lead confirms zero open Critical or High defects.

Prepared by (name/date)	Reviewed by (name/date)	Approved by (name/date)

Common mistakes

Common mistake	What to do instead
Working on a huge branch for weeks	Merge small changes daily; split tickets
'Works on my machine'	Test on staging; keep README and .env.example accurate
Fixing something not in the ticket	Write a new ticket; keep PRs focused
Trusting form validation only	Validate again on the server

Example (fictional training product SlotWise – numbers are illustrative)
SlotWise: 18 modules built over 11 weeks; 142 Pull Requests, all reviewed; weekly demos recorded; G6 signed.

Learn more (official sources – verify the page and note the date)

- React docs https://react.dev/learn
- Next.js docs https://nextjs.org/docs
- OWASP Cheat Sheets https://cheatsheetseries.owasp.org/

Stuck? Try 30 minutes alone → ask a colleague → ask the Tech Lead (technical) or Product Owner (scope). Bring the goal, what you tried, the exact error, a screenshot.

STAGE 7 / 12

Test It

VERIFY PHASE

G7 – Quality Approved

Goal: Find the problems before customers do, with proof that every requirement works.

Start when	Who is involved	Time	Tools	Skills needed
Features are on staging (testing runs alongside Stage 6 and finishes after G6).	QA Engineer (leads) · Developers (fix) · Product Owner (user acceptance)	2–3 weeks (overlaps Stage 6)	Test framework, Playwright (browser tests), Lighthouse, browsers and phones, bug tracker	Reading the PRD; patience.

Step by step

#	What to do	How to do it (exact)	Proof to save
1	**Write the test plan from the acceptance criteria**	A table: test ID · requirement · steps · expected result · result (pass/fail) · tester · date. Every acceptance criterion needs at least one test case.	Test plan sheet
2	**Check unit tests**	Developers' tests for single functions must pass in CI. QA reviews that business rules (calculations, validations) have tests with the hand-calculated examples.	CI green; coverage report
3	**Run integration tests**	Tests that call the API and use a test database: create → read → update → delete, permission checks, error responses.	Passing integration suite
4	**Run end-to-end (browser) tests for key journeys**	Automate the 5–10 most important journeys (sign-up, login, create booking, pay, export) with Playwright; run in CI against staging.	E2E results
5	**Do manual exploratory testing**	Try to break it: wrong inputs, empty fields, very long text, special characters, double-click submit, back button, slow network, two browser tabs. Test on Chrome, Edge, Firefox, Safari, and on at least one real Android and one iPhone.	Exploratory notes + bug list
6	**Check accessibility**	Use only the keyboard to complete a journey; run Lighthouse accessibility; check contrast and labels.	Accessibility result
7	**Check performance**	Run Lighthouse; test the main pages on a throttled mobile connection; do a simple load test with the expected peak requests per second from the cost model.	Performance numbers vs targets in the PRD
8	**Report bugs properly**	Use the bug template (Appendix A): title, environment, steps to reproduce, expected, actual, screenshot, severity. **Critical** = data loss/security/unusable; **High** = main feature broken; **Medium** = workaround exists; **Low** = cosmetic.	Tickets for every bug
9	**Retest and run regression**	When a bug is fixed retest it, then rerun the whole plan (regression) because fixes can break other things.	Updated results
10	**User acceptance test (UAT)**	The PO (and customer) run the sample scenarios from the PRD on staging and sign each as accepted.	UAT sign-off sheet
11	**Write the QA report and request G7**	Summary: tests run/passed/failed, open bugs by severity, performance and accessibility results, known limitations, recommendation.	G7 signed

Self-check before you ask for approval

- ☐ Every acceptance criterion has a test result
- ☐ No open Critical or High bugs
- ☐ Key journeys pass on 4 browsers and 2 phones
- ☐ Performance and accessibility targets met or exceptions approved
- ☐ UAT signed by PO
- ☐ QA report written

Sign-off: QA Lead and Product Owner sign G7. QA never tests their own code.

Prepared by (name/date)	Reviewed by (name/date)	Approved by (name/date)

Common mistakes

Common mistake	What to do instead
Testing only the happy path	Add invalid, empty, huge and duplicate inputs

Common mistake	What to do instead
Vague bug reports	Always include steps to reproduce
Skipping regression	Re-run the full plan after fixes

> **Example (fictional training product SlotWise – numbers are illustrative)**
> SlotWise: 212 test cases, 3 Critical bugs found in week 1 and fixed; final run 212/212 pass; G7 signed.

Learn more (official sources – verify the page and note the date)

- Playwright https://playwright.dev/docs/intro
- Lighthouse https://developer.chrome.com/docs/lighthouse/overview

Stuck? Try 30 minutes alone → ask a colleague → ask the Tech Lead (technical) or Product Owner (scope). Bring the goal, what you tried, the exact error, a screenshot.

ed by V.LOKESHWARAN · velsoftwaresolutions.in@g
VEL
SOFTWARE SOLUTIONS

STAGE 8 / 12

Secure It

VERIFY PHASE

G8 – Security Approved

Goal: Make sure we do not leak data, allow break-ins, or ship known vulnerabilities.

Start when	Who is involved	Time	Tools	Skills needed
G6 signed; QA in progress.	Tech Lead (owns) · Developers (fix) · DevOps · optional external tester	3–5 days	npm audit, GitHub security tabs, browser dev tools, OWASP checklists	Understanding of the app's roles and data.

Step by step

#	What to do	How to do it (exact)	Proof to save
1	**Scan for vulnerable libraries**	Run `npm audit` and read GitHub's Dependabot alerts. Update or replace packages with High/Critical findings; record any exception with a reason and date.	Audit output saved
2	**Scan for leaked secrets**	Check GitHub secret-scanning alerts and search the code for keys or passwords. If any real secret was ever committed, **replace it** (changing the code alone is not enough).	Zero open alerts
3	**Test access control (OWASP #1)**	Log in as customer A and try to open customer B's record by changing the ID in the address or API call. Log in as each role and try forbidden actions. All must be refused **by the server**.	Test table: role × action × result
4	**Test injection and input handling**	Type quotes, scripts and very long text into every field; confirm the app shows them safely and never errors. Check the code uses parameterised queries from the ORM, never string-built SQL.	Results noted per form
5	**Test login security**	Passwords hashed; limit repeated login attempts; sessions expire; logout really logs out; password-reset links expire and work once.	Auth checklist ticked
6	**Check HTTPS and headers**	Only HTTPS works (HTTP redirects); cookies are Secure and HttpOnly; basic security headers set; no secrets in browser source or API responses.	Header check output
7	**Check files and uploads**	Only allowed types and sizes; stored privately; no executable files; links expire.	Upload tests
8	**Check logs and errors**	Logs contain no passwords, keys or personal data; users never see stack traces.	Log sample reviewed
9	**Check backups and restore**	Restore the latest backup into staging and compare row counts. A backup never restored is only a hope.	Restore evidence
10	**Privacy basics**	Collect only needed data; record consent where needed; define retention and deletion; write the privacy policy with a lawyer. Confirm legal duties (for example India's Digital Personal Data Protection Act, 2023) with the company's legal adviser.	Privacy checklist + legal note
11	**Fix, retest and write the Security Report**	Fix all Critical/High items, retest, then write: what was checked, findings, fixes, residual risk, recommendation. Request G8.	G8 signed

Self-check before you ask for approval

- ☐ No High/Critical vulnerable libraries
- ☐ No secrets in code or history
- ☐ Cross-customer access impossible (tested)
- ☐ Every role tested for forbidden actions
- ☐ HTTPS only; cookies secure
- ☐ Backup restore proven
- ☐ Privacy policy and consent reviewed by legal adviser

Sign-off: Tech Lead signs G8; Owner accepts any residual risk in writing.

Prepared by (name/date)	Reviewed by (name/date)	Approved by (name/date)

Common mistakes

Common mistake	What to do instead
Hiding a mistake you made	Report it at once; the fix is quick, the cover-up is not

Common mistake	What to do instead
Relying on the front end to hide buttons	Hidden is not secure; check on the server
Never testing restore	Do it before every launch

Example (fictional training product SlotWise – numbers are illustrative)
SlotWise: cross-tenant test found one endpoint missing a tenant filter; fixed and covered by an automatic test; G8 signed.

Learn more (official sources – verify the page and note the date)

- OWASP Top 10 https://owasp.org/www-project-top-ten/
- OWASP ASVS https://asvs.owasp.org/
- NIST SSDF https://csrc.nist.gov/pubs/sp/800/218/final

Stuck? Try 30 minutes alone → ask a colleague → ask the Tech Lead (technical) or Product Owner (scope). Bring the goal, what you tried, the exact error, a screenshot.

ed by V.LOKESHWARAN · velsoftwaresolutions.in@g
VEL
SOFTWARE SOLUTIONS

STAGE 9 / 12

Work Out Cost and Price

LAUNCH PHASE

G9 – Cost & Price Approved

Goal: Know what the product truly costs to build and run, and set a price that leaves a profit after every cost.

Start when	Who is involved	Time	Tools	Skills needed
G6 signed (hours known) – can start earlier with estimates.	Owner / Finance (owns) · Tech Lead (usage numbers) · DevOps (provider prices) · Sales (price feedback)	1–2 weeks	Cost & Pricing workbook (Appendix B is a worked example), provider pricing pages and calculators	Basic spreadsheet formulas.

Step by step

#	What to do	How to do it (exact)	Proof to save
1	**Collect the hours**	From timesheets get hours per role and activity. For future products estimate three numbers: optimistic (O), most likely (M), pessimistic (P). Expected = (O + 4M + P) ÷ 6.	Hours table
2	**Calculate development labour**	Labour cost = hours × **fully loaded hourly rate** of that role (pay + employer costs + equipment + software seats + office, divided by productive hours). Include founder time. Show optimistic, expected, pessimistic totals.	Labour table (Appendix B.1)
3	**List one-time setup costs**	Domain, legal documents, payment-gateway onboarding, trademark, penetration test. Mark each **Mandatory** or **Optional**. One-time is never mixed with monthly.	Setup table
4	**List every monthly cost**	Domain (annual ÷ 12), hosting, database, storage, CDN, email, SMS/WhatsApp, authentication, outside APIs, AI services, monitoring, backups, security tools, payment fees, maintenance hours, support hours, miscellaneous.	Monthly cost table
5	**Get real prices from official pages**	For each provider open its **official pricing page or calculator**; copy plan, unit, price, currency, region and **date visited** into the Source column. If you cannot get a price, write ASSUMPTION in red. **Never guess.** Free tiers can change; never treat them as permanent.	Price list with links and dates
6	**Write the usage assumptions**	Registered users → monthly active (MAU) → daily active (DAU) → requests per user per day → peak requests per second → storage per user → bandwidth → emails/messages. Use low/medium/high if you have no data.	Assumption table
7	**Find the real bottleneck**	For 10, 50, 100, 500, 1,000, 5,000 and 10,000 users compare the load with plan limits (requests/second, database size, bandwidth). Cost the first limit you hit. Do **not** just multiply by number of users.	Capacity table
8	**Cost three architectures**	**Lean** (low cost MVP), **Professional** (paid, backups, monitoring), **Scalable** (redundancy, scaling). For each give setup, monthly, maintenance, annual, capacity, limits, risks and when to upgrade.	Three-model table
9	**Compute totals with the standard formulas**	One-time = research + design + development + testing + setup + deployment + legal. Monthly operating = hosting + database + storage + bandwidth + APIs + email + monitoring + backups + maintenance + support + other. Annual = monthly × 12. **First-year = one-time + annual.** Count shared company costs once; allocate by time share.	Totals box
10	**Set the price three ways**	**Cost-based**: (cost to serve + fixed share) ÷ (1 − margin). **Competitor-based**: position inside the range from your Stage 1 sheet. **Value-based**: a fair share of the saving the customer gets. Choose a price inside all three.	Price reasoning page
11	**Define the plans**	Starter, Professional, Business, Enterprise (and a Free Trial if suitable). For each: monthly price, annual price, users, storage, API limit, features, support level, overage charge, upgrade rule. Never promise 'unlimited' without a cost and abuse limit.	Plan table
12	**Check profit, not revenue**	Compute: contribution per customer = price − refunds − failed payments − gateway fee − variable delivery cost − support. **Break-even customers = fixed monthly cost ÷ contribution** (round up). Also CAC, LTV = contribution ÷ churn, LTV:CAC (target ≥ 3), payback (target < 12 months). Show tax separately; GST collected is not revenue.	Break-even and profit table
13	**Review and request G9**	Present to the Owner. Mark every figure as **Measured** or **Assumption**. Update the model whenever scope or provider prices change.	G9 signed

Self-check before you ask for approval

☐ Every price has a link and date or is marked ASSUMPTION
☐ One-time and monthly costs are separate
☐ Maintenance and support labour included
☐ Shared costs counted once
☐ Capacity bottleneck identified
☐ LTV:CAC ≥ 3 and payback < 12 months
☐ Profit shown after all costs; tax shown separately

Sign-off: Owner and Finance sign G9. Prices are published only after G9.

Prepared by (name/date)	Reviewed by (name/date)	Approved by (name/date)

Common mistakes

Common mistake	What to do instead
Quoting only domain + hosting	Include labour, support, maintenance, payment fees, legal
Using a competitor's price blindly	Triangulate with cost and value
Forgetting refunds, failed payments, GST	Use leakage factors; GST is pass-through

Example (fictional training product SlotWise – numbers are illustrative)
SlotWise worked example (Appendix B): expected development labour ₹10,88,000; break-even 28 paying clinics.

Learn more (official sources – verify the page and note the date)

- AWS Pricing Calculator https://calculator.aws/
- Vercel pricing https://vercel.com/pricing
- Cloudflare plans https://www.cloudflare.com/plans/
- CBIC GST https://cbic-gst.gov.in/

Stuck? Try 30 minutes alone → ask a colleague → ask the Tech Lead (technical) or Product Owner (scope). Bring the goal, what you tried, the exact error, a screenshot.

STAGE 10 / 12

Put It Online

LAUNCH PHASE

G10 – Go-Live Approved

Goal: Take the tested product from the repository to a real address, safely, with monitoring and a way back.

Start when	Who is involved	Time	Tools	Skills needed
G7, G8 and G9 are signed.	DevOps (executes) · Tech Lead (verifies) · Owner (owns accounts, approves go-live)	2–4 days	Registrar, hosting platform, DNS panel, terminal (dig, curl), uptime monitor, error tracker	Stage 5 complete; patience with details.

Step by step

#	What to do	How to do it (exact)	Proof to save
1	**Choose and check domain names**	Make 5–10 ideas. Search each at 2–3 reputable registrars to see **real** availability and compare first-year and renewal prices. Search trademark databases for conflicts. Never tell the customer a name is available until you checked that day.	Domain shortlist with prices and dates
2	**Register in the company's name**	Use the company account (not personal). Turn on MFA, auto-renew, registrar lock and privacy. Record: registrar, owner, renewal date, recovery method, who has access. Put the renewal date in a shared calendar with a 60-day reminder.	Domain/DNS Report started
3	**Create the production environment**	On the hosting platform create the production project, database, file storage; enter production secrets in the platform's secret settings (not in code). Production uses its own database, never staging's.	Environment ready; secrets stored
4	**Deploy to staging first and verify**	Connect the repository, set build settings, deploy to staging, open it and run the smoke test list (below).	Staging passes smoke tests
5	**Run database migrations on production**	Back up first, then run migrations through the pipeline, never by hand editing tables.	Migration log
6	**Add the domain on the host and copy exact DNS records**	In the host's domain settings add your domain. The host shows **the exact records** to create. Copy them character by character. Never invent a value. Before changing anything, export or screenshot the existing DNS zone so email records are preserved.	Screenshot of current zone + required records
7	**Create the records at your DNS provider**	Add the records the host gave you. Lower the TTL (e.g. 300 seconds) a day before the change so mistakes fix quickly.	Records saved
8	**Check DNS propagation**	In a terminal: `dig +short example.com A`, `dig +short www.example.com CNAME`, `dig +short example.com MX`, `dig +short example.com TXT`. Compare with what you entered. Propagation can take minutes to hours.	Command outputs saved
9	**Confirm HTTPS and redirect**	The host issues the certificate automatically after DNS works. Check `curl -I http://example.com` shows a redirect (301/308) to https, and `curl -I https://example.com` shows 200.	curl outputs saved
10	**Set up email authentication**	Add SPF, DKIM and DMARC as given by your email provider. Send a test to a mailbox and view 'original message' to confirm SPF, DKIM and DMARC say **pass**.	Headers show pass
11	**Switch on backups, monitoring, error tracking and alerts**	Daily database backup with a tested restore; uptime check every 1–5 minutes from two places; error tracker connected; alerts go to the on-call person's phone and email.	Test alert received
12	**Run production smoke tests (15 minutes)**	Sign up, log in, create and read the main record, pay with a test card then one real small payment and refund it, receive an email, export a file, log out. Any failure = stop and fix.	Smoke test sheet all green
13	**Test the rollback**	On staging redeploy the previous version and reverse the last migration in under 15 minutes. Record the exact steps.	Rollback evidence
14	**Complete the Deployment Report and request G10**	Fill the template with: version, date, who deployed, env settings (names only), test results, rollback proof. Owner approves go-live.	G10 signed

DNS records in plain English

Record	What it does (simple)	Rule
A / AAAA	Points a name to a server's IPv4 / IPv6 address	Copy exact value from the host

Record	What it does (simple)	Rule
CNAME	'This name is an alias of that name'	Cannot sit beside other records on the same name
MX	Where email for the domain is delivered	Copy from the email provider; never overwrite existing MX
TXT	Text proofs: ownership, SPF, DKIM, DMARC	Only one SPF record per domain
SPF	Which servers may send your email	Start from the provider's include value
DKIM	Digital signature on your emails	Publish selector exactly as given
DMARC	What receivers do with unauthenticated mail	Start with p=none + reports, tighten after review
CAA	Which companies may issue certificates	Must include your host's CA

Self-check before you ask for approval

☐ Domain owned by the company with MFA and auto-renew
☐ Existing email records preserved
☐ DNS values exactly match the host's instructions
☐ HTTPS valid; HTTP redirects to HTTPS
☐ SPF, DKIM, DMARC pass
☐ Backup restore tested
☐ Alerts reach a real person
☐ Smoke tests green; rollback proven

Sign-off: Tech Lead verifies; Owner signs G10. Never go live on a Friday evening.

Prepared by (name/date)	Reviewed by (name/date)	Approved by (name/date)

Common mistakes

Common mistake	What to do instead
Overwrote MX records and email stopped	Always export the zone first; restore the saved MX records
Certificate not issued	DNS is wrong or not propagated yet; re-check records and wait
Registered in an employee's personal account	Transfer to the company account immediately

Example (fictional training product SlotWise – numbers are illustrative)
SlotWise: domain registered, DNS verified with dig, HTTPS redirect confirmed, rollback tested in 9 minutes; G10 signed on a Tuesday morning.

Learn more (official sources – verify the page and note the date)

- Cloudflare: what is DNS https://www.cloudflare.com/learning/dns/what-is-dns/
- ICANN https://www.icann.org/
- Vercel domains https://vercel.com/docs/domains

Stuck? Try 30 minutes alone → ask a colleague → ask the Tech Lead (technical) or Product Owner (scope). Bring the goal, what you tried, the exact error, a screenshot.

STAGE 11 / 12

Show, Sell and Deliver

LAUNCH PHASE

G11 – Customer Acceptance

Goal: Win paying customers honestly, onboard them well and hand the product over with written acceptance.

Start when	Who is involved	Time	Tools	Skills needed
Staging or production demo is stable; G9 signed (price known).	Sales Lead · Product Owner · Support Lead · Owner (approves terms) · Legal adviser	2–8 weeks per customer	Demo environment with fake data, proposal and SOW templates, e-signature, CRM/pipeline in the dashboard	Clear speaking; honesty about limits.

Step by step

#	What to do	How to do it (exact)	Proof to save
1	**Prepare demo data**	Create a clean demo account with **fake or written-authorised data**. Never show real customer data.	Demo account ready
2	**Write the demo script (25 minutes)**	1 Introduction · 2 Their problem · 3 How they do it today · 4 Our solution · 5 Live workflow · 6 Benefits · 7 **Limitations (say them honestly)** · 8 Pricing · 9 Pilot proposal · 10 Next steps. Show only finished features.	Script
3	**Rehearse and record**	Practise twice with a colleague; fix anything that fails. Keep a screen recording as backup.	Rehearsal done
4	**Find and qualify leads**	Use your ICP from Stage 1. A good lead has the problem, budget, a decision-maker and a reason to act now. Log every lead in the pipeline with next action and date.	Pipeline updated
5	**Hold a discovery call**	Listen first: current process, pain, volume, budget, decision process, timeline. Write notes within 24 hours.	Discovery notes
6	**Give the demo**	Follow the script; ask for questions; note objections.	Demo notes
7	**Send the proposal and Statement of Work**	Proposal: problem, solution, price, timeline. SOW: exact scope, deliverables, acceptance criteria, timeline, responsibilities, fees. Legal adviser reviews contract terms: ownership of code and reusable components, third-party licences, customer data ownership, confidentiality, liability, termination, data export, support hours.	Documents sent; Owner approved terms
8	**Run the pilot**	30–60 days with 2–3 success measures agreed in writing; weekly check-ins; fix issues quickly.	Pilot report
9	**Onboard the customer**	Create their tenant, import data, set roles, train users (e.g. 2 × 90 minutes), hand over the **user manual** and **admin manual**, share support contacts and hours.	Onboarding checklist
10	**Get written acceptance**	The customer signs the Acceptance Form confirming the SOW criteria are met. Issue the invoice per the agreement.	Signed acceptance
11	**Handover pack**	Manuals, training slides, support process, escalation contacts, subscription terms, renewal date. Ask permission in writing before using their name, logo, screenshot or quote anywhere.	G11 signed

Self-check before you ask for approval

☐ No unfinished feature was shown as finished
☐ Demo used fake or authorised data
☐ Contract defines ownership of code, data and IP
☐ Pilot measures written and met
☐ Customer signed acceptance
☐ Written consent held before any logo or testimonial use

Sign-off: Customer signs acceptance; Owner signs G11.

Prepared by (name/date)	Reviewed by (name/date)	Approved by (name/date)

Common mistakes

Common mistake	What to do instead
Promising future features	Say 'not available today' and offer a Change Request
Verbal scope agreements	Put everything in the SOW
Using real data in a demo	Use fake data or get written authorisation

Example (fictional training product SlotWise – numbers are illustrative)
SlotWise: 3 clinics piloted for 60 days at an agreed pilot price; all 3 signed acceptance; 2 converted to annual plans.

Learn more (official sources – verify the page and note the date)

- MeitY https://www.meity.gov.in/

Stuck? Try 30 minutes alone → ask a colleague → ask the Tech Lead (technical) or Product Owner (scope). Bring the goal, what you tried, the exact error, a screenshot.

velsoftwaresolutions.in@gmail.com · Developed by V.LOKESHWARAN · VEL SOFTWARE SOLUTIONS

STAGE 12 / 12

Keep It Running

OPERATE PHASE

G12 – Operations Accepted

Goal: Keep the service up, secure, supported and profitable, because subscriptions only continue if the product works every day.

Start when	Who is involved	Time	Tools	Skills needed
Product is live (G10) and customers are onboarded.	Support Lead · DevOps · Tech Lead (incident commander) · Owner (reviews monthly)	Ongoing	Uptime monitor, error tracker, logs, ticket system, status page, dashboard	Calm communication; reading logs.

Step by step

#	What to do	How to do it (exact)	Proof to save
1	**Daily (15 min)**	Check the dashboard: uptime, errors, backup success, new tickets. Reply to every new ticket (first response target agreed in the contract).	Daily check ticked
2	**Weekly (1 hour)**	Review error trends, slow pages, failed payments, expiring certificates/domains; plan patches; update the board.	Weekly note
3	**Monthly: patch and update**	Update dependencies and apply security patches on staging, test, then deploy through the pipeline.	Patch list
4	**Monthly: restore drill**	Restore the latest backup to staging and compare data. Record time taken.	Drill evidence
5	**Monthly: cost and usage review**	Compare real invoices with the cost model; check usage vs plan limits; flag customers near limits for upgrade; update the model.	Cost review sheet
6	**Monthly: subscriptions and billing**	Check renewals due, failed payments, refunds, churn; contact at-risk customers; confirm invoices and GST records with the accountant.	MRR, churn, renewals sheet
7	**Handle support tickets**	Acknowledge → classify severity → fix or workaround → confirm with customer → close with notes. Repeated questions become help articles.	Ticket history
8	**Handle incidents using the matrix**	**Critical**: service down or serious security incident – Tech Lead leads, tell the Owner at once, post status update, fix or roll back, review within 5 days. **High**: major feature unavailable. **Medium**: limited impact, workaround exists. **Low**: cosmetic. Each incident gets an owner, updates to customers, and a written post-incident review.	Incident report
9	**Handle feature requests**	Never build them straight away: log them, the PO scores value/effort, accepted ones go through Change Request or the next release plan.	Request log
10	**Write the monthly Maintenance Report**	Uptime, incidents, patches, backup drill, cost vs plan, support statistics, customer satisfaction, next month's actions. Send to Owner and customer if contracted.	Report saved in 10-maintenance
11	**Review the service promise**	Never promise an SLA (uptime or response time) the team cannot staff and monitor. Review quarterly.	G12 signed

Incident levels

Level	Meaning	Owner	First response (example)	Customer message
Critical	Service down or confirmed serious security incident	Tech Lead	≤ 30 min if 24×7 is staffed	Status page + email at once, update every hour
High	Major function unavailable	DevOps on call	≤ 2 working hours	Email, update every 4 hours
Medium	Limited function, workaround exists	Support → Developer	≤ 1 working day	Ticket updates
Low	Minor or cosmetic	Support Lead	≤ 3 working days	Ticket

Example response times only. Publish a target only if staffing and monitoring can meet it.

Self-check before you ask for approval

☐ Monitoring and alerts are working
☐ Backups restored this month
☐ Patches applied within the agreed window
☐ Costs compared with the model

☐ Incidents all have reports
☐ Nothing promised that we cannot deliver

Sign-off: Tech Lead signs G12 when runbooks, on-call and monitoring exist; Owner reviews monthly.

Prepared by (name/date)	Reviewed by (name/date)	Approved by (name/date)

Common mistakes

Common mistake	What to do instead
Fixing production directly	Always use the pipeline; hotfix branch + review even when urgent
No one watches alerts	Define an on-call rota with phone numbers
Silent outage	Tell customers early; silence destroys trust

Example (fictional training product SlotWise – numbers are illustrative)
SlotWise: monthly report shows 99.9% measured uptime, 2 Medium incidents, backup drill 22 minutes, infrastructure cost 6% under plan.

Learn more (official sources – verify the page and note the date)

- Google SRE book https://sre.google/sre-book/table-of-contents/
- NIST SSDF https://csrc.nist.gov/pubs/sp/800/218/final

Stuck? Try 30 minutes alone → ask a colleague → ask the Tech Lead (technical) or Product Owner (scope). Bring the goal, what you tried, the exact error, a screenshot.

APPENDIX A

Forms and templates to fill

A.1 Gate sign-off sheet (one per project – print and keep in 02-projects/.../)

Gate	Stage	Evidence attached	Self-check done	Peer review by	Approver	Date	Signature
G1	Check the idea	☐	☐		Owner		
G2	Write what we build	☐	☐		PO + Customer		
G3	Plan how it works	☐	☐		Tech Lead + Owner		
G4	Design screens	☐	☐		PO + Customer		
G5	Set up project	☐	☐		Tech Lead		
G6	Build it	☐	☐		PO + Tech Lead		
G7	Test it	☐	☐		QA Lead + PO		
G8	Secure it	☐	☐		Tech Lead (+Owner for risk)		
G9	Cost & price	☐	☐		Owner + Finance		
G10	Put it online	☐	☐		Tech Lead + Owner		
G11	Show, sell, deliver	☐	☐		Customer + Owner		
G12	Keep it running	☐	☐		Tech Lead		

A.2 Project record (create on day 1 of every project)

Field	Value
Project ID (P001)	
Product name	
Product Owner	
Tech Lead	
Developers	
QA owner	
Customer contact	
Budget (₹)	
Planned delivery date	
Approved scope version	
Production cost estimate	
Sales price	
Expected margin %	
Current stage / last gate signed	
Risk register link	

A.3 Pull Request template

```
## What changed
(one or two sentences)

## Why
Ticket: #__

## How to test
1. ...
2. ...

## Screenshots (if screen changed)

## Checklist
- [ ] Tests added/updated and passing
- [ ] Inputs validated on the server
- [ ] No secrets or customer data
- [ ] Docs updated
- [ ] Time logged
```

A.4 Bug report template

Field	Content
Title	Short and specific
Environment	staging / production · browser · device
Steps to reproduce	1. 2. 3.
Expected result	
Actual result	
Screenshot / video	
Severity	Critical / High / Medium / Low
Found by / date	

A.5 Change Request form

Field	Content
CR ID	CR-P001-001
Requested by / date	
Description	
Business reason	
Technical impact	
Cost impact (₹)	
Timeline impact (days)	
Security impact	
Customer approval	Name / date / signature
Product Owner decision	Approved / Rejected / Later

A.6 Test case template

ID	Requirement	Steps	Expected	Actual	Pass/Fail	Tester / date
TC-001	PRD 4.2 booking conflict	1 Book 10:15. 2 Book 10:15 again	Second booking refused; next 3 slots offered			
TC-002						

A.7 Weekly report (every Friday)

Field	Content
Name / week	
Completed (ticket links)	

Field	Content
Pending	
Blockers (what, since when, who can help)	
Hours by project ID and stage	
Quality issues found	
Next week's deliverables	

A.8 Deployment checklist

Check	Done
Version tagged and release notes written	☐
Database backed up	☐
Migrations run on production	☐
Environment settings checked (names only)	☐
DNS verified with dig	☐
HTTPS and redirect verified with curl	☐
Email SPF/DKIM/DMARC pass	☐
Smoke tests green	☐
Monitoring and alert test received	☐
Rollback steps ready	☐
Owner approval recorded (G10)	☐

A.9 Handover checklist

Item	Done
User manual and admin manual delivered	☐
Training sessions completed and attendance recorded	☐
Support contacts, hours and escalation sent	☐
Acceptance form signed	☐
Invoice issued	☐
Subscription terms and renewal date recorded	☐
Consent obtained before any logo/testimonial use	☐
Credentials transferred only via the password manager	☐

A.10 Incident report

Field	Content
Incident ID / level	
Start and end time	
What happened	
Customer impact	
Root cause	
Fix applied	
How we prevent it	
Owner · reviewed by · date	

APPENDIX B

Worked cost, price and profit model (sample product)

Used in Stage 9. It shows **how** to calculate; replace every ASSUMPTION with a cited, dated official figure.

This section implements Phases 7 and 8 for the sample product. **Every unit price is an ASSUMPTION** until replaced by a cited official price.

A. One-time development cost – labour

`Development Labour Cost = Estimated Hours × Fully Loaded Hourly Cost`; Expected hours use PERT = (O + 4M + P) ÷ 6. Founder time is included.

Role	Fully loaded ₹/hour (ASSUMPTION)
Product Owner / Founder	₹1,200
Tech Lead	₹1,500
Full-stack Developer	₹800
UI/UX Designer	₹700
QA Engineer	₹500
DevOps Engineer	₹900

Fully loaded = salary or founder-equivalent pay + employer taxes/benefits + equipment + software seats + office/overheads, divided by productive (not paid) hours. Replace with your actuals.

Activity	Role	O h	Expected h (PERT)	P h	₹/h	Optimistic	Expected	Pessimistic
Market research	Product Owner / Founder	30	41.7	60	₹1,200	₹36,000	₹50,000	₹72,000
Requirement analysis & PRD	Product Owner / Founder	40	61.7	90	₹1,200	₹48,000	₹74,000	₹1,08,000
Requirement analysis & PRD	Tech Lead	10	16.3	24	₹1,500	₹15,000	₹24,500	₹36,000
UI/UX design	UI/UX Designer	80	113.3	160	₹700	₹56,000	₹79,333	₹1,12,000
Frontend development	Full-stack Developer	200	286.7	400	₹800	₹1,60,000	₹2,29,333	₹3,20,000
Backend development	Full-stack Developer	220	310.0	440	₹800	₹1,76,000	₹2,48,000	₹3,52,000
Database & API design	Tech Lead	40	61.7	90	₹1,500	₹60,000	₹92,500	₹1,35,000
Authentication & RBAC	Full-stack Developer	30	41.7	60	₹800	₹24,000	₹33,333	₹48,000
Integrations (payments, email, SMS)	Full-stack Developer	40	63.3	100	₹800	₹32,000	₹50,667	₹80,000
Testing	QA Engineer	100	146.7	220	₹500	₹50,000	₹73,333	₹1,10,000
Security review	Tech Lead	16	25.3	40	₹1,500	₹24,000	₹38,000	₹60,000
Deployment & DevOps	DevOps Engineer	24	38.0	60	₹900	₹21,600	₹34,200	₹54,000
Documentation	Full-stack Developer	24	38.0	60	₹800	₹19,200	₹30,400	₹48,000
Customer onboarding	Product Owner / Founder	16	25.3	40	₹1,200	₹19,200	₹30,400	₹48,000
Total		**870**	**1270**	**1844**		**₹7,41,000**	**₹10,88,000**	**₹15,83,000**

B. One-time infrastructure & setup cost

Item	Amount (ASSUMPTION)	Type	Note
Domain registration (year 1)	₹1,000	Mandatory	Assumption; check registrar price page for the exact TLD
Legal: T&Cs, Privacy Policy, DPA review	₹25,000	Mandatory	Assumption; obtain lawyer quote
Hosting / DB / storage / SSL / DNS setup	₹0	Mandatory	Labour already in DevOps hours; provider setup fees assumed nil
Payment-gateway onboarding	₹0	Mandatory	Assumed nil setup fee; verify on gateway pricing page
Trademark search & filing	₹8,000	Optional	Optional; assumption
External penetration test	₹40,000	Optional	Optional but recommended before enterprise sales; assumption

Item	Amount (ASSUMPTION)	Type	Note
Total mandatory	**₹26,000**		
Total optional	**₹48,000**		

C/D. Usage assumptions and capacity scenarios

Assumption	Value
Monthly active users (of registered)	80%
Daily active users (of registered)	50%
Requests per DAU per day	150
Share of daily requests in busiest hour	15%
Burst factor on peak hour	3×
Peak concurrency	10% of DAU
DB queries per request	4
Storage per user	50 MB
Bandwidth per DAU per day	3 MB
Emails per user per month	40
SMS/WhatsApp per user per month	60

Low/High scenarios: halve or double the request, storage and messaging values; re-run the model.

Users	MAU	DAU	Requests/day	Peak req/s (3× burst)	Peak concurrent	DB queries/day	Storage GB	Bandwidth GB/mo	Emails/mo	SMS+WA msgs/mo
10	8	5	750	0.1	0	3,000	0.5	0	400	600
50	40	25	3,750	0.5	2	15,000	2.4	2	2,000	3,000
100	80	50	7,500	0.9	5	30,000	4.9	4	4,000	6,000
500	400	250	37,500	4.7	25	150,000	24.4	22	20,000	30,000
1,000	800	500	75,000	9.4	50	300,000	48.8	44	40,000	60,000
5,000	4,000	2,500	375,000	46.9	250	1,500,000	244.1	220	200,000	300,000
10,000	8,000	5,000	750,000	93.8	500	3,000,000	488.3	439	400,000	600,000

Unit-rate assumptions

Unit rate (ALL ASSUMPTIONS)	Value
Object storage	₹4 / GB-month
Bandwidth / CDN egress	₹8 / GB
Transactional email	₹80 / 1,000 emails
SMS / WhatsApp	₹0.5 / message
Maintenance labour	₹850 / hour
Support labour	₹600 / hour; 0.5 h per tenant per month
Lean MVP: base platform	₹2,500 / month incl. app host, managed DB, backups, monitoring
Lean MVP: extra compute unit	₹1,500 per unit; one unit sustains 10 req/s
Professional: base platform	₹9,000 / month incl. app host, managed DB, backups, monitoring
Professional: extra compute unit	₹4,000 per unit; one unit sustains 40 req/s
Scalable: base platform	₹35,000 / month incl. app host, managed DB, backups, monitoring
Scalable: extra compute unit	₹8,000 per unit; one unit sustains 100 req/s

Replace before use
Every rate above is a planning placeholder to make the model computable. Before Gate G7, replace each with the provider's current official price (URL + access date) using the Cost Estimation template. Do not quote customers from these numbers.

E. Three production models – monthly operating cost by user count

Cost is bottleneck-driven: compute is stepped by peak requests/second (units needed = ⌈peak ÷ unit capacity⌉), storage/bandwidth/messages scale with usage, maintenance is a fixed labour block, support scales with tenants (users ÷ 7.3 per clinic). Not a straight line.

Lean MVP

Users	Compute units	Platform + compute	Usage (storage, BW, email, msgs)	Maintenance labour	Support labour	Monthly operating	Annual	Status
10	1	₹2,500	₹337	₹10,200	₹411	**₹13,448**	₹1,61,381	OK
50	1	₹2,500	₹1,687	₹10,200	₹2,055	**₹16,442**	₹1,97,306	OK
100	1	₹2,500	₹3,375	₹10,200	₹4,110	**₹20,184**	₹2,42,211	OK
500	1	₹2,500	₹16,873	₹10,200	₹20,548	**₹50,121**	₹6,01,457	OK
1,000	1	₹2,500	₹33,747	₹10,200	₹41,096	**₹87,543**	₹10,50,513	OK
5,000	5	₹8,500	₹1,68,734	₹10,200	₹2,05,479	**₹3,92,914**	₹47,14,966	Outgrown – upgrade
10,000	10	₹16,000	₹3,37,469	₹10,200	₹4,10,959	**₹7,74,628**	₹92,95,532	Outgrown – upgrade

Professional

Users	Compute units	Platform + compute	Usage (storage, BW, email, msgs)	Maintenance labour	Support labour	Monthly operating	Annual	Status
10	1	₹9,000	₹337	₹17,000	₹411	**₹26,748**	₹3,20,981	OK
50	1	₹9,000	₹1,687	₹17,000	₹2,055	**₹29,742**	₹3,56,906	OK
100	1	₹9,000	₹3,375	₹17,000	₹4,110	**₹33,484**	₹4,01,811	OK
500	1	₹9,000	₹16,873	₹17,000	₹20,548	**₹63,421**	₹7,61,057	OK
1,000	1	₹9,000	₹33,747	₹17,000	₹41,096	**₹1,00,843**	₹12,10,113	OK
5,000	2	₹13,000	₹1,68,734	₹17,000	₹2,05,479	**₹4,04,214**	₹48,50,566	OK
10,000	3	₹17,000	₹3,37,469	₹17,000	₹4,10,959	**₹7,82,428**	₹93,89,132	OK

Scalable

Users	Compute units	Platform + compute	Usage (storage, BW, email, msgs)	Maintenance labour	Support labour	Monthly operating	Annual	Status
10	1	₹35,000	₹337	₹34,000	₹411	**₹69,748**	₹8,36,981	OK
50	1	₹35,000	₹1,687	₹34,000	₹2,055	**₹72,742**	₹8,72,906	OK
100	1	₹35,000	₹3,375	₹34,000	₹4,110	**₹76,484**	₹9,17,811	OK
500	1	₹35,000	₹16,873	₹34,000	₹20,548	**₹1,06,421**	₹12,77,057	OK
1,000	1	₹35,000	₹33,747	₹34,000	₹41,096	**₹1,43,843**	₹17,26,113	OK
5,000	1	₹35,000	₹1,68,734	₹34,000	₹2,05,479	**₹4,43,214**	₹53,18,566	OK
10,000	1	₹35,000	₹3,37,469	₹34,000	₹4,10,959	**₹8,17,428**	₹98,09,132	OK

Model comparison

Model	Typical build	Estimated capacity	Limitations / risks	Upgrade triggers
Lean / Low-cost MVP	Managed PaaS free/low tier + small managed Postgres, daily backup, basic uptime + error tracking	≈ up to 1,000 users (peak < 10 req/s)	Free tiers can change or be withdrawn; shared resources; limited backups; single region; no redundancy	Paying customers > 25, any SLA promise, peak load > 70% of unit, DB > 70% of storage cap
Professional	Paid PaaS/VM, managed Postgres with point-in-time recovery, staged environments, monitoring, security scanning	≈ up to 5,000 users with extra compute units	Single region; manual failover; cost grows stepwise	Enterprise customers, uptime > 99.5% contractually, p95 latency breaches, DB CPU > 60%
Scalable	Multi-AZ app + DB, autoscaling, CDN/WAF, centralised logs, DR in second region, 24×7 alerting	10,000+ users	Highest fixed cost and operations load; needs on-call staff	Revenue/MRR > monthly cost × 3 and enterprise commitments

Model (at 1,000 users)	One-time setup (non-labour)	Monthly infrastructure	Monthly maintenance	Monthly operating (incl. support)	Annual recurring
Lean MVP	₹26,000	₹36,247	₹10,200	₹87,543	₹10,50,513
Professional	₹26,000	₹42,747	₹17,000	₹1,00,843	₹12,10,113
Scalable	₹26,000	₹68,747	₹34,000	₹1,43,843	₹17,26,113

F. Complete cost formula

```
Total One-Time Cost   = Research + Design + Development + Testing + Setup + Deployment + Legal/Compliance
Monthly Operating Cost= Hosting + Database + Storage + Bandwidth + APIs + Email + Monitoring + Backups +
Maintenance + Support + Other
Annual Operating Cost = Monthly Operating Cost × 12
Total First-Year Cost = Total One-Time Cost + Annual Operating Cost
```

Item	Amount
Total one-time cost (labour expected + mandatory setup)	₹11,14,000
Monthly operating cost @100 users, Professional	₹33,484
Annual operating cost = monthly × 12	₹4,01,811
Total first-year cost = one-time + annual operating	**₹15,15,811**
Range using pessimistic labour	₹20,10,811
Range using optimistic labour	₹11,68,811

Shared vs product-specific: Shared company items (GitHub org, design tool seats, password manager, accounting software) are totalled once and allocated to products by employee-time share (example: ₹10,000 shared × 25% = ₹2,500 charged to the product). Product-specific items (its hosting, DB, domain, SMS) are charged 100%. A cost line is never both allocated and charged directly.

Pricing & Profitability (Phase 8)

Pricing approaches

Approach	Method	Result for sample
Cost-based	(Cost to serve + share of fixed cost) ÷ (1 − target margin)	Starter cost to serve ₹401 + fixed share at 100 tenants ₹416 = ₹817 break-even price; with 30% margin ≈ ₹1,167 (before gateway/leakage) – so ₹999 is a thin-margin entry plan, upsell to Professional
Competitor-based	Place inside the observed range from ≥5 competitor pricing pages	*Pending research (no competitor prices invented)*
Value-based	10–20% of customer's quantified saving (e.g. recovered no-show revenue)	If a clinic recovers 5 visits/month × ₹500 = ₹2,500, a ₹999 plan captures ~40%: validate in discovery

Plans

Plan	Monthly (ex-GST)	Annual (10× monthly = 2 months free)	Users	Storage	API calls/mo	Support	Cost to serve + leakage	Contribution / month	Contribution %
Free Trial	₹0 (14 days)	–	2	0.5 GB	2,000	Docs only	₹31 variable (excl. support time)	–	Counted in CAC
Starter	₹999	₹9,990	3	2 GB	10,000	Email, 2 business days	₹470	₹529	53%
Professional	₹2,499	₹24,990	8	10 GB	50,000	Email + chat, 1 business day	₹742	₹1,757	70%
Business	₹5,999	₹59,990	20	50 GB	250,000	Priority email/chat, 4 business hours	₹1,689	₹4,310	72%
Enterprise	Custom (from ≈ ₹15,000)	Custom	Custom	Custom	Custom	Named contact, SLA by contract	Quote per deal	≥ 60% target	Contract

Overage (all plans): extra user ₹150/month; extra 10 GB ₹100/month; extra 10,000 API calls ₹200; SMS/WhatsApp at provider cost + 20%. Upgrade condition: usage > 80% of any limit for 2 consecutive months triggers an upgrade prompt. All prices are ASSUMPTIONS to be validated in discovery.

Fixed monthly cost allocated to the product

Fixed monthly cost (allocated to product)	₹ / month
Professional-model hosting & DB base	₹9,000
Maintenance labour (20 h x ₹850)	₹17,000
Domain (annualised)	₹83
Accounting / legal retainer (allocated)	₹3,000
Shared dev tools (allocated by time share 25% of ₹10,000)	₹2,500
Fixed marketing	₹10,000
Monthly fixed operating cost	**₹41,583**

Key metrics

`CAC = S&M acquisition cost ÷ new paying customers` · `MRR = Σ active subscriptions` · `ARR = MRR × 12` · `Gross margin % = (Revenue − Cost of revenue) ÷ Revenue × 100` · `Contribution = Revenue − variable delivery costs` · `Break-even = Fixed ÷ Contribution per customer (round up)` · `Operating profit = Revenue − all operating expenses`. Cost of revenue here = platform hosting base + maintenance labour + variable delivery + support + gateway fees + refunds/failed-payment leakage; sales/marketing and G&A are operating expenses.

Metric	Value
Blended ARPA (plan mix 50/35/15)	₹2,274 / tenant / month
Contribution per tenant (after gateway fees, refunds, failed payments, variable delivery, support)	₹1,526
Monthly fixed operating cost	₹41,583
Break-even customers = fixed ÷ contribution = ₹41,583 ÷ ₹1,526	**27.25 → round UP to 28 paying tenants**
Churn (ASSUMPTION)	4% / month → lifetime 25 months
LTV = contribution × lifetime	₹38,144
CAC (ASSUMPTION; = sales+marketing spend ÷ new paying customers)	₹6,000
LTV : CAC	6.4 : 1 (target ≥ 3)
CAC payback	3.9 months (target < 12)
Maximum acceptable CAC (LTV ÷ 3)	₹12,715
Revenue leakage assumed	refunds 2% + failed payments 3%
Payment gateway fee (ASSUMPTION; verify gateway page)	2% of collected amount
GST on subscriptions	18% collected from customer, remitted to government – NOT revenue (confirm rate/place-of-supply with CA & CBIC)
Monthly revenue target for 12-month goal	₹2,27,400 MRR (100 tenants)

Profitability by paying-tenant count

Paying tenants	MRR	ARR	Contribution	Operating profit (before tax)	Income tax @25.17% (ASSUMPTION; confirm with CA)	Profit after tax	Gross margin
10	₹22,740	₹2,72,880	₹15,257	-₹26,326	₹0	-₹26,326	-47%
25	₹56,850	₹6,82,200	₹38,144	-₹3,440	₹0	-₹3,440	21%
28	₹63,672	₹7,64,064	₹42,721	₹1,137	₹286	₹851	26%
50	₹1,13,700	₹13,64,400	₹76,287	₹34,704	₹8,735	₹25,969	44%
100	₹2,27,400	₹27,28,800	₹1,52,574	₹1,10,991	₹27,936	₹83,054	56%
250	₹5,68,500	₹68,22,000	₹3,81,435	₹3,39,852	₹85,541	₹2,54,311	63%
500	₹11,37,000	₹1,36,44,000	₹7,62,871	₹7,21,288	₹1,81,548	₹5,39,739	65%

Tax is shown separately: the model applies a flat assumed rate only to positive operating profit and ignores losses carried forward, GST input credit and founder remuneration rules – have a chartered accountant confirm.

Commercial recommendation

Recommendation	Value
Initial setup / onboarding fee	₹5,000 (Starter) · ₹10,000 (Professional) · ₹25,000 (Business) – covers data import and training; recovers CAC
Monthly subscription	₹999 / ₹2,499 / ₹5,999 (ex-GST)
Annual subscription	10× monthly (2 months free); billed upfront – improves cash flow and reduces churn
Minimum viable selling price	Starter ₹999: contribution ₹529 = 53% of price; contribution is still positive at ₹799 but leaves almost no room for CAC, so do not discount below it
Target gross margin	≥ 60% – model reaches 56% at 100 tenants and 63% at 250 tenants; below 60% until roughly 175 tenants
Maximum CAC	₹12,715 (LTV ÷ 3)
Break-even customers	28 paying tenants
Renewal strategy	90/60/30-day renewal reminders; annual plans auto-renew with notice; quarterly business review for Business/Enterprise; price rises ≤ 10% with 60-day notice

App store fees (mobile distribution)

Applies only if the product is also distributed as an iOS or Android app and sells digital goods or subscriptions inside the app. A browser-based B2B SaaS billed through your own gateway has no store commission, but check each store's current rules before deciding the billing channel. Fees below were checked on **2026-11-03**.

Apple – verified from official Apple pages
Fetched directly on 2026-11-03 from developer.apple.com/programs/whats-included and developer.apple.com/app-store/small-business-program.

Item	Official wording / value	Notes
Developer Program fee	"99 USD per membership year, or in local currency where available"	The page gives no INR amount; read the local price at enrolment. Fee waiver exists for eligible nonprofits, accredited educational institutions and government entities.
Standard commission	30% on sale of digital goods and services	Page states 30%.
Small Business Program	15% on paid apps and In-App Purchases	Eligible if proceeds ≤ USD 1 million in the prior calendar year (all associated accounts) and ≤ USD 1 million in the current year; above that the standard rate applies to future sales; can re-qualify the following year. Rate change applies 15 days after the end of the fiscal month of approval.
Qualifying subscriptions	15%	Page lists 15% for qualifying subscriptions.
10% rate	"further reduced commission of 10%" for EU alternative-terms developers in the Small Business Program and for subscriptions after their first year	The sentence is ambiguous about who gets 10%; confirm on the page before modelling it.
Regional variations	Different rates/fees may apply in Brazil, EU, Japan, Netherlands, Russia, South Korea	India is not in that list on the page checked.

Google Play – NOT directly verified
Google's help pages (support.google.com, play.google.com) could not be opened from this build environment (network block). The figures below come from web-search summaries of Google's own pages and **must be confirmed on the linked page before use**. Google has changed its fee model in 2026 and the sources below describe it differently by region and date.

Item	Reported (search summary, 2026-11-03)	Verify at
Developer registration	One-time USD 25 (reported on Google's EEA access terms page)	Play Console Help / Play Console sign-up page
Older structure	15% on the first USD 1M of annual earnings, 30% above; 15% on auto-renewing subscriptions	Service fees

Item	Reported (search summary, 2026-11-03)	Verify at
2026 structure	Fee depends on whether the user's install is 'new' or 'existing' relative to a regional start date; first USD 1M at 10%; auto-renewing subscriptions 10%; other in-app purchases reported at 20% (new) / 25% (existing); billing fee reported at 5% where Google Play Billing is used (US/UK/EEA)	Understanding Google Play's lower service fees; Android Developers Blog
Reported start dates	30 Jun 2026: EEA, UK, US · 30 Sep 2026: Australia, Japan · 31 Dec 2026: South Korea · 30 Sep 2027: rest of world	Same page. **If correct, India (rest of world) would still be on the older structure until the 2027 date** – confirm.
Programs	Apps Experience / Games Level Up programs reported to give lower rates from 30 Sep 2026 in AU, EEA, JP, UK, US	Play Console Help

Effect on the sample plans (illustrative)

Contribution if the same plan were sold in-app, replacing the gateway fee with a store commission on the full price. Uses the same variable-cost assumptions as the financial model; ignores tax handling differences, which vary by store and country.

Plan	Price	Contribution via own gateway	In-app at 30%	In-app at 15%	Impact
Starter	₹999	₹529	₹248	₹398	13% lower at 15% vs web
Professional	₹2,499	₹1,757	₹1,054	₹1,429	13% lower at 15% vs web
Business	₹5,999	₹4,310	₹2,624	₹3,524	13% lower at 15% vs web

Decision rules

1. Prefer web billing for B2B subscriptions where the store's rules allow it; confirm the current rules for your category.
2. If in-app sales are required, price the in-app plan so contribution stays ≥ the web plan's margin target, or accept a lower margin knowingly.
3. Enrol in Apple's Small Business Program as soon as eligible; check Google programs for your region.
4. Budget fixed store costs: Apple USD 99/year (local price at enrolment) and Google USD 25 one-time (to confirm). Add them to the one-time/annual cost lines when mobile apps are in scope.
5. Re-verify all store fees at every G7 and G8 review.

APPENDIX C

Quick-reference commands

I want to...	Command
Where am I?	`pwd` (Windows PowerShell: `Get-Location`)
List files	`ls` / `dir`
Go into / out of a folder	`cd folder` / `cd ..`
Clone a repository	`git clone git@github.com:ORG/repo.git`
See what changed	`git status` · `git diff`
Update main	`git switch main && git pull`
New branch	`git switch -c feature/12-short-name`
Save changes	`git add .` then `git commit -m "feat: what changed"`
Send to GitHub	`git push -u origin feature/12-short-name`
Undo uncommitted edits to a file	`git restore path/to/file`
See history	`git log --oneline -10`
Install / run / test / build	`npm ci` · `npm run dev` · `npm test` · `npm run build`
Lint	`npm run lint`
Check vulnerable libraries	`npm audit`
DNS lookups	`dig +short example.com A` · `dig +short example.com MX` · `dig +short example.com TXT`
Check HTTPS redirect	`curl -I http://example.com` · `curl -I https://example.com`
Check a port is answering	Open the URL in the browser or `curl -I http://localhost:3000`

Before you press Enter
If a command deletes, resets or overwrites something, stop and ask the Tech Lead. Never run commands you do not understand, especially ones copied from the internet or containing 'sudo' or 'force'.

ed by V.LOKESHWARAN · velsoftwaresolutions.in@g
VEL
SOFTWARE SOLUTIONS

APPENDIX D

The 20 standard documents and the management dashboard

#	Template	Purpose	Mandatory fields
1	Market Research Report	Opportunity evidence	Segment, problem, TAM/SAM/SOM workbook, sources with access dates, recommendation, kill criteria
2	Competitor Analysis	Compare 5–10 competitors	Name, URL, access date, pricing, features, tech signals, reviews, positioning, gaps
3	Customer Discovery	Interview log	Interviewee role, date, current workflow, pain rank, spend today, willingness to pay, quotes, consent
4	PRD	Product definition	Purpose, customers, roles/permissions, modules, journeys, NFRs, backup, maintenance
5	Input/Output Specification	Data contract	Per field: type, unit, required, default, validation, min/max, source, DB map; per output: format, logic, precision, display, export, API shape, errors
6	Scope Freeze	Baseline + change control	Version, approvers, out-of-scope, CR log (ID, justification, tech/cost/timeline/security impact, approvals)
7	Architecture Document	Technical design	Options scored, diagrams, ERD, API, auth, threat model, infrastructure, ADRs
8	UI/UX Approval	Design sign-off	Sitemap, flows, hi-fi links, design tokens, usability findings, approvals
9	GitHub Setup	Repo checklist	Repo URL, branch rules, CI checks, secrets storage, environments, release tags
10	Development Report	Weekly progress	Modules done, requirement refs, hours vs estimate, demo notes, blockers
11	Code Review	PR review record	Scope, correctness, security, tests, readability, decision
12	QA Report	Test evidence	Test plan, cases run/passed/failed, defects by severity, regression, UAT sign-off
13	Security Report	Pre-launch review	ASVS checklist, dependency scan, secrets scan, auth/RBAC tests, findings, residual risk
14	Cost Estimation	Phase 7 workbook	Labour three-point, setup, monthly, capacity, three models, source URLs + dates
15	Pricing Model	Phase 8 workbook	Three pricing approaches, plans, margins, break-even, CAC/LTV, assumptions
16	Domain/DNS Report	Domain record	Registrar, owner, MFA, renewal date, DNS records (exact values), propagation proof
17	Deployment Report	Release record	Build, env vars (names only), migrations, smoke tests, rollback proof, approver
18	Customer Showcase	Demo pack	Script, dataset, limitations list, pricing, pilot proposal, next steps
19	Project Handover	Delivery close	Deliverables, credentials transfer procedure, manuals, training, acceptance form
20	Maintenance Report	Monthly ops	Uptime, incidents, patches, backup test, cost vs budget, support stats, actions

Each template is stored in the dashboard Documents module, versioned, and linked to its gate. The Change Request, Weekly Status and Risk Register forms are below.

Internal management dashboard (for managers)

Modules

#	Module	Purpose	Primary roles
1	Company Overview	KPIs, alerts, pending approvals	All
2	Project Management	Project record, stage tracker, timeline	PM, Tech Lead
3	Market Research	Reports, competitor sheets, sources	Product
4	Product Requirements	PRD/I-O versions	Product
5	Scope Approval	Freeze + CR workflow	Product, Owner
6	Architecture Approval	Docs + ADR + approval	Tech Lead
7	UI/UX Approval	Design links + sign-off	Design, Product
8	Employee Assignments	Roster, capacity	PM
9	Development Tracking	Tasks, hours vs estimate	Dev
10	GitHub Repository Tracking	Repos, PRs, CI status, releases	Tech Lead
11	QA & Testing	Test runs, defects	QA
12	Production Cost Calculator	Phase 7 model	Finance
13	SaaS Pricing Calculator	Phase 8 model	Finance, Owner
14	Domain & DNS Tracker	Domains, renewals, records	DevOps

#	Module	Purpose	Primary roles
15	Deployment Tracker	Releases, rollbacks	DevOps
16	Customer Management	Accounts, contacts, consents	Sales, Support
17	Sales Pipeline	Stages, forecasts	Sales
18	Subscription Management	Plans, renewals, MRR	Finance, Support
19	Maintenance Management	Tickets, incidents, SLA	Support, DevOps
20	Reports & Analytics	Exports, trend charts	Managers

Management KPIs

KPI	Definition
Total / active / delayed projects	Count from project records; delayed = current date > planned date of current stage
Project budgets vs actual expenditure	Budget field vs timesheet hours × loaded rate + invoices
Estimated vs actual production cost	Phase 7 model vs provider invoices
Expected revenue	Σ weighted pipeline value (stage probability × ACV)
MRR	Σ active subscription price (ex-GST, net of discounts)
Gross margin	(Revenue − cost of revenue) ÷ Revenue; cost of revenue defined in pricing model
Employee productivity	Delivered story points ÷ hours; never used alone for appraisal
Customer satisfaction	CSAT/NPS from post-onboarding and quarterly surveys
Pending approvals	Open gate/CR requests per approver

Roles and permissions

Role	Can do
Employee	Update tasks, log time, upload deliverables, request approvals
Lead	Review, approve technical gates, assign work
Manager / Owner	Approve, reject or request changes at all gates; see finance
Finance	Edit cost/pricing models, view revenue
Admin	Users, roles, audit log

Each project shows a stage tracker G1 → G12. Dashboard is responsive, role-based (server-side checks), and audit-logged. It is built as Project 0 using this same operating system.

APPENDIX E

Source index and assumption register

Source	URL	Verified on (fill)
VS Code	https://code.visualstudio.com/docs	___/___/_____
GitHub	https://docs.github.com/	___/___/_____
Next.js	https://nextjs.org/docs	___/___/_____
React	https://react.dev/	___/___/_____
Node.js	https://nodejs.org/docs/	___/___/_____
PostgreSQL	https://www.postgresql.org/docs/	___/___/_____
Vercel	https://vercel.com/docs	___/___/_____
Cloudflare	https://developers.cloudflare.com/	___/___/_____
AWS	https://docs.aws.amazon.com/	___/___/_____
Google Cloud	https://cloud.google.com/docs	___/___/_____
Microsoft Azure	https://learn.microsoft.com/azure/	___/___/_____
OWASP	https://owasp.org/	___/___/_____
OWASP ASVS	https://asvs.owasp.org/	___/___/_____
NIST SSDF SP 800-218	https://csrc.nist.gov/pubs/sp/800/218/final	___/___/_____
ICANN	https://www.icann.org/	___/___/_____
MeitY	https://www.meity.gov.in/	___/___/_____
CBIC (GST)	https://cbic-gst.gov.in/	___/___/_____

Source	URL	Verified on (fill)
WCAG 2.2	https://www.w3.org/TR/WCAG22/	___/___/_____
Google SRE book	https://sre.google/sre-book/table-of-contents/	___/___/_____
Apple Developer Program – what's included	https://developer.apple.com/programs/whats-included/	**2026-11-03** – page fetched
Apple App Store Small Business Program	https://developer.apple.com/app-store/small-business-program/	**2026-11-03** – page fetched
Google Play service fees	https://support.google.com/googleplay/android-developer/answer/112622	2026-11-03 – search summary only; page blocked, re-verify
Google Play lower service fees (2026)	https://support.google.com/googleplay/android-developer/answer/16954621	2026-11-03 – search summary only; page blocked, re-verify
Android Developers Blog – expanded billing	https://android-developers.googleblog.com/2026/06/play-expanded-billing.html	2026-11-03 – search summary only; re-verify

Verification required
Links are authoritative starting points. Pricing, tax rates and regulations change; whoever uses a figure must open the page, confirm it, and record the date. Only the two Apple pages marked above were fetched directly; Google Play figures are search-reported and flagged as such; no cloud-provider price is quoted as fact.

Assumption register

ID	Assumption	Where used	Replace with
A1	Fully loaded hourly rates	Labour cost	HR/finance actuals
A2	Hours per activity (O/M/P)	Labour cost	Past project timesheets
A3	Provider unit prices	Infrastructure model	Official pricing pages + dates
A4	Usage per user (DAU, requests, MB)	Capacity	Analytics from pilot
A5	Plan prices & mix	Pricing	Customer discovery + competitor pages
A6	Churn, refunds, failed-payment rates	LTV, profit	Billing data after 3 months
A7	Gateway fee	Contribution	Gateway contract
A8	GST 18%, corporate tax 25.17%	Pricing, profit	CA / CBIC / Income-tax dept
A9	CAC ₹6,000	LTV:CAC	Actual sales & marketing spend

Developed by **V.LOKESHWARAN** · velsoftwaresolutions.in@gmail.com · VEL Software Solutions
© VEL Software Solutions. Internal handbook – all rights reserved.